**ĐẠI HỌC QUỐC GIA HÀ NỘI**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

## KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

# PHÁT HIỆN BẤT THƯỜNG TRÊN DỮ LIỆU CẢM BIẾN IOT BẰNG KIẾN TRÚC HỌC SÂU NHẸ

Đồ án tốt nghiệp ngành Khoa học máy tính

**Ngành: Khoa học máy tính**

**Cán bộ hướng dẫn: TS. Trần Minh Khoa**
**Sinh viên thực hiện: Nguyễn Văn An**
Mã số sinh viên: 20201234

**HÀ NỘI - 2026**

# LỜI CAM ĐOAN

Tôi xin cam đoan đây là công trình nghiên cứu do tôi thực hiện dưới sự hướng dẫn của TS. Trần Minh Khoa. Các số liệu, kết quả nêu trong báo cáo là trung thực, được tính toán từ mã nguồn và dữ liệu công khai; các kết quả chưa từng được công bố trong bất kỳ công trình nào khác. Những tài liệu, mã nguồn và ý tưởng tham khảo đều được trích dẫn rõ ràng.

Hà Nội, ngày 15 tháng 12 năm 2026

**Sinh viên thực hiện**

Nguyễn Văn An

# TÓM TẮT

Sự phổ biến của cảm biến IoT tạo ra dòng dữ liệu nhiều kênh, liên tục và có nhiễu. Phát hiện bất thường trên dòng dữ liệu này thường phải chạy tại biên vì độ trễ, băng thông và quyền riêng tư, nhưng các mô hình học sâu hiện đại vượt quá khả năng của vi điều khiển. Báo cáo đề xuất một kiến trúc lai CNN–LSTM–Autoencoder có attention, huấn luyện chỉ trên dữ liệu bình thường, và lượng tử hóa int8 để triển khai trên thiết bị biên.

Kiến trúc gồm ba phần: encoder CNN 1D học đặc trưng cục bộ, LSTM học phụ thuộc thời gian, attention chọn các bước quan trọng; decoder tái tạo cửa sổ đầu vào. Điểm bất thường là hợp nhất của sai số tái tạo và khoảng cách Mahalanobis trong không gian latent, với ngưỡng $\mu + k \cdot \sigma$ hiệu chỉnh trên tập validation. Mô hình được đánh giá trên NAB, SWaT, SMAP/MSL và một bộ dữ liệu IoT tổng hợp, so sánh với Isolation Forest, OCSVM, LSTM-AE, USAD, OmniAnomaly và một MLP autoencoder nhỏ.

Kết quả cho thấy mô hình đạt F1 0,89 và AUC-PR 0,91, cao hơn baseline tốt nhất 0,03–0,05; sau lượng tử hóa int8, F1 giảm 0,008 nhưng độ trễ còn 11,7 ms và RAM còn 64 KB trên STM32H747. Phân tích ablation cho thấy attention đóng góp 0,020 F1, Mahalanobis 0,015 F1. Báo cáo cũng phân tích bốn nhóm lỗi và đề xuất biện pháp giảm báo động giả do drift.

Đóng góp của báo cáo gồm ba phần: kiến trúc lai có attention, quy trình lượng tử hóa và hiệu chỉnh ngưỡng, và bộ đánh giá bốn bộ dữ liệu với phân tích lỗi. Mã nguồn và cấu hình được công bố kèm phụ lục để tái lập. Kết quả cho thấy mô hình nhỏ có thể là lựa chọn thực tế cho giám sát cảm biến tại biên, miễn là ngưỡng và lượng tử hóa được xử lý cùng nhau.

***Từ khóa:*** IoT, phát hiện bất thường, autoencoder, CNN–LSTM, TinyML

# MỤC LỤC

Mở đầu ..... 1
1 Tổng quan và công trình liên quan ..... 4
1.1 IoT và dữ liệu cảm biến ..... 4
1.2 Bài toán phát hiện bất thường ..... 4
1.2.1 Phân loại theo cấu trúc dữ liệu ..... 5
1.3 Các hướng tiếp cận ..... 5
1.3.1 Phương pháp thống kê ..... 5
1.3.2 Học máy cổ điển ..... 6
1.3.3 Học sâu ..... 6
1.3.4 Học sâu trên thiết bị biên ..... 6
1.4 Công trình liên quan ..... 7
1.5 Khoảng trống và hướng giải quyết ..... 8
1.6 Kết luận chương ..... 8
2 Cơ sở lý thuyết ..... 8
2.1 Dữ liệu chuỗi thời gian cảm biến ..... 8
2.2 Autoencoder ..... 9
2.2.1 Hạn chế của autoencoder ..... 9
2.3 CNN một chiều ..... 10
2.4 LSTM ..... 10
2.5 Attention ..... 11
2.6 Lượng tử hóa và triển khai biên ..... 11
2.7 Độ phức tạp tính toán ..... 11
2.8 Chỉ số đánh giá ..... 12
2.9 Ký hiệu ..... 13
2.10 Kết luận chương ..... 13
3 Phương pháp đề xuất ..... 14

3.1 Kiến trúc tổng thể ..... 14
3.2 Tiền xử lý và đặc trưng ..... 15
3.3 Nhánh encoder CNN–LSTM ..... 16
3.4 Decoder và điểm bất thường ..... 16
3.5 Huấn luyện ..... 17
3.6 Lượng tử hóa và triển khai ..... 17
3.7 Quy trình triển khai firmware ..... 18
3.8 Kiểm thử trên ESP32-S3 ..... 19
3.9 Tham số và độ phức tạp ..... 19
3.10 Kết luận chương ..... 19
4 Thực nghiệm và đánh giá ..... 20
4.1 Bộ dữ liệu ..... 20
4.2 Thiết lập thực nghiệm ..... 20
4.3 Kết quả chính ..... 21
4.4 So sánh với baseline ..... 23
4.5 Ablation ..... 23
4.6 Độ trễ và bộ nhớ ..... 24
4.7 Độ nhạy của ngưỡng ..... 25
4.8 So sánh với công trình ..... 26
4.9 Phân tích lỗi ..... 27
4.10 Thảo luận ..... 27
4.11 Chi phí triển khai ..... 28
4.12 Đe dọa đến tính hợp lệ ..... 29
4.13 Kết luận chương ..... 30
5 Kết luận và hướng phát triển ..... 30
5.1 Kết quả đạt được ..... 30
5.2 Hạn chế ..... 31
5.3 Bài học thiết kế ..... 31

5.4 Tác động xã hội và đạo đức ..... 31
5.5 Hướng phát triển ..... 32
5.6 Kết luận ..... 33
Phụ lục A Cấu hình huấn luyện và triển khai ..... 34
Phụ lục B Kết quả chi tiết theo lớp cảm biến ..... 35
TÀI LIỆU THAM KHẢO ..... 36

# DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

| Ký hiệu | Diễn giải |
|---|---|
| **AE** | Autoencoder |
| **AUC-PR** | Area Under the Precision–Recall Curve |
| **CNN** | Convolutional Neural Network |
| **FAR** | False Alarm Rate |
| **FN** | False Negative |
| **FP** | False Positive |
| **F1** | F1-score |
| **IoT** | Internet of Things |
| **LSTM** | Long Short-Term Memory |
| **MCU** | Microcontroller Unit |
| **MSE** | Mean Squared Error |
| **MTTD** | Mean Time To Detect |
| **NAB** | Numenta Anomaly Benchmark |
| **QAT** | Quantization-Aware Training |
| **SMAP** | Soil Moisture Active Passive |
| **MSL** | Mars Science Laboratory |
| **SWaT** | Secure Water Treatment |
| **TN** | True Negative |
| **TP** | True Positive |
| **TinyML** | Machine Learning on Microcontrollers |

# DANH MỤC BẢNG

Bảng 1.1. Phân loại bất thường trong dữ liệu cảm biến ..... 5
Bảng 1.2. So sánh các công trình liên quan ..... 7
Bảng 2.1. Độ phức tạp của các toán tử ..... 12
Bảng 2.2. Các chỉ số đánh giá ..... 12
Bảng 2.3. Ký hiệu và đơn vị ..... 13
Bảng 3.1. Các nhóm đặc trưng đầu vào ..... 15
Bảng 3.2. Ngân sách bộ nhớ firmware ..... 18
Bảng 3.3. Độ phức tạp của các mô hình ..... 19
Bảng 4.1. Các bộ dữ liệu thực nghiệm ..... 20
Bảng 4.2. Cấu hình các baseline ..... 21
Bảng 4.3. Kết quả tổng hợp ..... 21
Bảng 4.4. F1 theo từng bộ dữ liệu ..... 23
Bảng 4.5. Kết quả ablation ..... 24
Bảng 4.6. Độ trễ và bộ nhớ trên STM32H747 ..... 25
Bảng 4.7. Độ nhạy của ngưỡng theo k ..... 26
Bảng 4.8. So sánh với công trình công khai ..... 26
Bảng 4.9. Phân tích lỗi ..... 27
Bảng 4.10. Confusion matrix trên tập test ..... 27
Bảng 5.1. Đối chiếu mục tiêu và kết quả ..... 30
Bảng A.1. Cấu hình huấn luyện và triển khai ..... 34
Bảng B.1. Kết quả theo lớp cảm biến ..... 35

# DANH MỤC HÌNH

Hình 3.1. Kiến trúc CNN–LSTM–Autoencoder có attention. .....14
Hình 3.2. Cửa sổ trượt và vùng bất thường. .....15
Hình 4.1. Đường cong huấn luyện và validation. .....22
Hình 4.2. Đường cong precision–recall. .....22
Hình 4.3. So sánh F1 của sáu phương pháp. .....23
Hình 4.4. Đóng góp của từng thành phần. .....24
Hình 4.5. Độ trễ suy luận trên STM32H747. .....25

# Mở đầu

Trong thập niên vừa qua, cảm biến giá rẻ và kết nối không dây đã đưa IoT từ phòng thí nghiệm ra nhà máy, nông trại, bệnh viện và hộ gia đình. Một hệ thống giám sát nhỏ có thể sinh ra hàng chục nghìn điểm dữ liệu mỗi ngày; một nhà máy có hàng trăm cảm biến có thể sinh ra hàng triệu điểm. Dữ liệu này có giá trị chỉ khi hệ thống phát hiện được sự kiện bất thường: một cảm biến hỏng, một van bị kẹt, một đợt tấn công mạng, hay một thay đổi môi trường làm hỏng sản phẩm.

Phát hiện bất thường là bài toán xác định các điểm hoặc đoạn dữ liệu khác biệt có ý nghĩa so với hành vi bình thường. Với dữ liệu cảm biến, bài toán khó ở ba điểm. Thứ nhất, nhãn bất thường hiếm và đắt; phần lớn hệ thống chỉ có dữ liệu bình thường. Thứ hai, dữ liệu nhiều kênh, có tương quan và phụ thuộc thời gian; một ngưỡng đơn lẻ trên từng kênh bỏ sót bất thường tập thể. Thứ ba, mô hình phải chạy gần nguồn dữ liệu để giảm độ trễ và băng thông, nhưng vi điều khiển chỉ có vài trăm KB RAM và không có GPU.

Các hướng tiếp cận học sâu như autoencoder, LSTM và Transformer đã đạt kết quả tốt trên dữ liệu chuỗi thời gian, nhưng thường được đánh giá trên máy chủ. Khoảng trống giữa độ chính xác và khả năng triển khai biên là động lực của đề tài này. Báo cáo đặt câu hỏi: có thể thiết kế một mô hình đủ nhẹ để chạy trên vi điều khiển mà vẫn giữ được độ chính xác cạnh tranh với các mô hình nặng hơn hay không?

## Tính cấp thiết

Trong công nghiệp, một cảnh báo muộn có thể dẫn đến dừng dây chuyền, hỏng thiết bị hoặc tai nạn. Trong nông nghiệp, độ ẩm và nhiệt độ sai lệch có thể làm mất mùa. Trong y tế tại nhà, nhịp tim hoặc nồng độ oxy bất thường cần được phát hiện trong vài giây. Những tình huống này không cho phép gửi toàn bộ dữ liệu lên cloud rồi chờ kết quả: độ trễ, chi phí và quyền riêng tư đều là ràng buộc thực.

Một mô hình chạy trên thiết bị biên còn giúp hệ thống hoạt động khi mất mạng, giảm dữ liệu thô phải truyền và cho phép phản hồi cục bộ. Tuy nhiên, thiết bị biên có giới hạn về năng lượng, bộ nhớ và khả năng tính toán. Vì vậy, bài toán không chỉ là tối ưu độ chính xác mà là tối ưu đồng thời độ chính xác, độ trễ và bộ nhớ.

## Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát là thiết kế, huấn luyện và đánh giá một kiến trúc học sâu nhẹ cho phát hiện bất thường trên dữ liệu cảm biến IoT đa biến, có thể lượng tử hóa và triển khai trên vi điều khiển. Bốn câu hỏi nghiên cứu cụ thể là:

- **RQ1:** Kiến trúc lai CNN–LSTM–Autoencoder có attention có thể học biểu diễn bình thường từ dữ liệu đa biến mà không cần nhãn bất thường hay không?
- **RQ2:** Kiến trúc đề xuất so sánh với các baseline thống kê, học máy cổ điển và học sâu như thế nào trên bốn bộ dữ liệu?
- **RQ3:** Lượng tử hóa int8 ảnh hưởng đến độ chính xác, độ trễ và bộ nhớ ra sao khi triển khai trên STM32H747?
- **RQ4:** Những nhóm lỗi nào chiếm ưu thế, và biện pháp nào có thể giảm báo động giả mà không giảm recall?

## Đối tượng và phạm vi

Đối tượng nghiên cứu là dữ liệu chuỗi thời gian đa biến từ cảm biến IoT, với giả thiết huấn luyện chỉ có dữ liệu bình thường. Phạm vi gồm bốn bộ dữ liệu: NAB, SWaT, SMAP/MSL và một bộ dữ liệu tổng hợp 12 kênh. Báo cáo không xét dữ liệu hình ảnh, âm thanh, văn bản hay chuỗi sự kiện rời rạc. Báo cáo cũng không xét học trực tuyến trên thiết bị; mô hình được huấn luyện ngoại tuyến và triển khai suy luận.

Về phần cứng, thực nghiệm dùng STM32H747 Discovery và ESP32-S3 để đo độ trễ, nhưng chỉ STM32H747 được báo cáo chi tiết vì có bộ nhớ lớn hơn. Kết quả trên ESP32-S3 được dùng để kiểm tra tính khả chuyển, không dùng để so sánh tuyệt đối.

## Phương pháp nghiên cứu

Báo cáo kết hợp bốn phương pháp: khảo sát tài liệu để xác định khoảng trống; thiết kế kiến trúc theo hướng design science; thực nghiệm định lượng với 5 seed và khoảng tin cậy 95%; phân tích lỗi định tính trên các cửa sổ sai. Mỗi thí nghiệm có baseline, cùng cách chia dữ liệu và cùng quy trình chọn ngưỡng để bảo đảm so sánh công bằng.

Các chỉ số chính là precision, recall, F1, AUC-PR, FAR và MTTD. Độ trễ được đo bằng đồng hồ phần cứng, mỗi cấu hình 1.000 lần suy luận sau 100 lần khởi động. Bộ nhớ được đo bằng linker map và bộ đếm heap của FreeRTOS.

## Đóng góp

- **Kiến trúc:** một autoencoder lai CNN–LSTM–attention gọn, huấn luyện chỉ trên dữ liệu bình thường, với điểm bất thường hợp nhất sai số tái tạo và Mahalanobis.
- **Quy trình lượng tử hóa:** QAT cho mô hình chuỗi thời gian, hiệu chỉnh ngưỡng sau lượng tử hóa, và kiểm tra trên hai vi điều khiển.
- **Benchmark:** so sánh 6 phương pháp trên 4 bộ dữ liệu với cùng giao thức, kèm phân tích ablation và phân tích lỗi.
- **Mã nguồn và cấu hình:** cấu hình huấn luyện, mã xuất TFLite Micro và script tái lập kết quả được để trong phụ lục.

## Bố cục báo cáo

Báo cáo gồm năm chương. Chương 1 trình bày tổng quan về IoT, bài toán phát hiện bất thường và công trình liên quan. Chương 2 trình bày cơ sở lý thuyết về autoencoder, CNN, LSTM, attention, lượng tử hóa và chỉ số đánh giá. Chương 3 mô tả kiến trúc đề xuất và quy trình huấn luyện. Chương 4 trình bày thực nghiệm, so sánh, ablation, độ trễ và phân tích lỗi. Chương 5 kết luận và nêu hướng phát triển. Tài liệu tham khảo và hai phụ lục ở cuối báo cáo.

# 1 Tổng quan và công trình liên quan

## 1.1 IoT và dữ liệu cảm biến

IoT là mạng các thiết bị vật lý có khả năng cảm nhận, tính toán và truyền thông. Một nút cảm biến điển hình gồm vi điều khiển, một hoặc nhiều cảm biến, bộ nhớ, radio và nguồn. Dữ liệu được lấy mẫu theo chu kỳ, đóng gói và gửi lên gateway hoặc cloud. Tùy ứng dụng, tần số lấy mẫu có thể từ vài giây đến vài mili giây; số kênh có thể từ một đến hàng trăm.

Dữ liệu cảm biến có bốn đặc điểm ảnh hưởng trực tiếp đến bài toán phát hiện bất thường. Thứ nhất, dữ liệu có tính chuỗi: giá trị hiện tại phụ thuộc giá trị trước đó. Thứ hai, các kênh có tương quan: nhiệt độ và độ ẩm thường đi cùng nhau, dòng điện và mô-men xoắn có quan hệ phi tuyến. Thứ ba, dữ liệu có nhiễu và mất mát: cảm biến có thể trả về NaN, giá trị cực đại hoặc giá trị kẹt. Thứ tư, phân phối có thể thay đổi theo thời gian do mùa, hao mòn hoặc thay đổi quy trình, hiện tượng gọi là concept drift.

Ba đặc điểm đầu tạo ra nhu cầu về mô hình học biểu diễn thay vì ngưỡng đơn lẻ. Đặc điểm thứ tư tạo ra nhu cầu về hiệu chỉnh ngưỡng và đánh giá theo thời gian, không chỉ trên một tập test cố định. Một mô hình có F1 cao trên tập test nhưng không thích nghi với drift sẽ nhanh chóng báo động giả trong vận hành.

## 1.2 Bài toán phát hiện bất thường

Theo Chandola và cộng sự, bất thường có thể là điểm, ngữ cảnh hoặc tập thể [1] . Bất thường điểm là một quan sát đơn lẻ khác xa phân phối; bất thường ngữ cảnh là quan sát bình thường ở điều kiện này nhưng bất thường ở điều kiện khác; bất thường tập thể là một nhóm quan sát cùng lệch, dù từng điểm có thể nằm trong ngưỡng. Dữ liệu cảm biến thường chứa cả ba loại.

Về cách học, bài toán được chia thành có giám sát, bán giám sát và không giám sát. Có giám sát cần nhãn cho cả bình thường và bất thường; bán giám sát thường huấn luyện chỉ trên dữ liệu bình thường; không giám sát tìm cấu trúc bất thường mà không dùng nhãn. Trong công nghiệp, nhãn bất thường hiếm và không đầy đủ, nên hướng bán giám sát thực tế hơn. Báo cáo chọn hướng này: huấn luyện chỉ trên cửa sổ bình thường, phát hiện cửa sổ không khớp với mô hình.

Một khó khăn khác là đánh giá. Nếu bất thường kéo dài nhiều điểm, một cảnh báo đúng có thể được tính là nhiều true positive, làm F1 bị thổi phồng. Báo cáo dùng

nhãn theo cửa sổ và báo cáo cả phát hiện theo điểm lẫn theo đoạn; mọi bảng kết quả ghi rõ đơn vị để tránh so sánh sai.

### 1.2.1 Phân loại theo cấu trúc dữ liệu

Mỗi loại bất thường đòi hỏi một cách phát hiện khác nhau. Bảng 1.2 hệ thống hóa bốn loại và ánh xạ chúng vào kỹ thuật phù hợp. Việc chọn sai loại bất thường là nguyên nhân phổ biến khiến mô hình có F1 cao trên giấy nhưng thất bại khi triển khai.

Bảng 1.1. Phân loại bất thường trong dữ liệu cảm biến

| Loại | Định nghĩa | Cách phát hiện | Ví dụ |
|---|---|---|---|
| **Điểm** | Một quan sát lệch khỏi phân phối tại thời điểm t | Ngưỡng, autoencoder, Isolation Forest | Cảm biến hỏng, gai nhiễu |
| **Ngữ cảnh** | Giá trị bình thường ở điều kiện này nhưng bất thường ở điều kiện khác | Mô hình chuỗi, cửa sổ trượt | Nhiệt độ cao vào mùa hè |
| **Tập thể** | Một nhóm điểm cùng lệch, dù từng điểm nằm trong ngưỡng | Tương quan kênh, attention | Tấn công SWaT, rò rỉ van |
| **Drift** | Phân phối thay đổi chậm theo thời gian | Cập nhật thống kê, học trực tuyến | Hao mòn cảm biến, đổi mùa |

*Nguồn: Tổng hợp từ Chandola và cộng sự [1]*

Bất thường điểm dễ phát hiện nhất nhưng cũng dễ bị nhiễu đánh lừa. Bất thường ngữ cảnh cần mô hình có khái niệm thời gian, ví dụ nhiệt độ 35°C là bình thường vào mùa hè nhưng bất thường vào mùa đông. Bất thường tập thể khó nhất vì từng điểm có thể nằm trong ngưỡng; mô hình phải học tương quan giữa các kênh. Drift không phải bất thường tại một thời điểm mà là thay đổi phân phối theo thời gian, nên cần cơ chế cập nhật thay vì chỉ tăng ngưỡng.

Trong bốn bộ dữ liệu của báo cáo, NAB chủ yếu chứa bất thường điểm và ngữ cảnh; SWaT chứa bất thường tập thể do tấn công; SMAP/MSL chứa cả ba loại; bộ tổng hợp được thiết kế để kiểm tra từng loại riêng. Cách chọn này cho phép đánh giá mô hình trên phổ rộng, thay vì tối ưu cho một loại duy nhất.

## 1.3 Các hướng tiếp cận

### 1.3.1 Phương pháp thống kê

Các phương pháp thống kê như ARIMA, EWMA, kiểm soát quá trình bằng thống kê (SPC) mô hình hóa phân phối chuẩn và đánh dấu điểm nằm ngoài khoảng tin cậy. Chúng nhanh, dễ giải thích và phù hợp với tín hiệu một kênh. Tuy nhiên, chúng

giả định phân phối ổn định và tuyến tính, khó bắt tương quan phi tuyến giữa nhiều kênh. Với dữ liệu cảm biến có drift, ngưỡng cố định nhanh chóng lỗi thời.

### 1.3.2 Học máy cổ điển

Isolation Forest, One-Class SVM, Local Outlier Factor và kNN là những baseline mạnh cho dữ liệu nhiều chiều. Isolation Forest cô lập điểm bất thường bằng các cây ngẫu nhiên; OCSVM tìm biên bao quanh dữ liệu bình thường; LOF dựa trên mật độ cục bộ. Các phương pháp này không cần nhãn bất thường và chạy nhanh, nhưng cần đặc trưng thủ công và không mô hình hóa trực tiếp thứ tự thời gian. Với cửa sổ dài, số chiều tăng và chi phí suy luận tăng.

### 1.3.3 Học sâu

Autoencoder học ánh xạ nén và tái tạo; mẫu bình thường có sai số tái tạo thấp, mẫu bất thường có sai số cao. LSTM autoencoder thêm khả năng học phụ thuộc thời gian [2] ; VAE học phân phối xác suất và cho điểm bất thường có cơ sở xác suất; Transformer dùng self-attention để nắm phụ thuộc dài. Các mô hình này đạt kết quả tốt nhưng thường có hàng triệu tham số, khó chạy trên vi điều khiển.

Trong nhóm học sâu, hai hướng liên quan trực tiếp đến báo cáo là autoencoder chuỗi và mô hình lai. Malhotra và cộng sự dùng LSTM autoencoder để phát hiện bất thường trên ECG [2] . Hundman và cộng sự dùng LSTM dự báo và ngưỡng động cho telemetry vệ tinh [3] . Su và cộng sự đề xuất OmniAnomaly, kết hợp VAE và GRU, mạnh trên chuỗi dài nhưng nặng [4] . Audibert và cộng sự đề xuất USAD, hai autoencoder đối kháng, huấn luyện nhanh hơn nhưng vẫn cần nhiều bộ nhớ [5] .

### 1.3.4 Học sâu trên thiết bị biên

TinyML nghiên cứu cách đưa mô hình học sâu lên vi điều khiển có tài nguyên hạn chế. Ba kỹ thuật chính là lượng tử hóa, cắt tỉa và chưng cất tri thức. Lượng tử hóa chuyển trọng số từ float32 sang int8, giảm 4 lần bộ nhớ và tăng tốc nhờ phép nhân số nguyên; cắt tỉa loại bỏ trọng số ít quan trọng; chưng cất huấn luyện mô hình nhỏ từ mô hình lớn. Banbury và cộng sự đưa ra bộ benchmark MLPerf Tiny để so sánh độ trễ và năng lượng [6] ; Goyal và cộng sự tổng quan các kỹ thuật TinyML [12] .

Tuy nhiên, phần lớn công trình TinyML tập trung vào phân loại ảnh hoặc từ khóa, không phải phát hiện bất thường trên chuỗi thời gian. Bài toán chuỗi có thêm hai thách thức: trạng thái ẩn của LSTM phải được quản lý giữa các cửa sổ, và ngưỡng phải hiệu chỉnh sau lượng tử hóa vì phân phối sai số thay đổi. Báo cáo giải quyết cả hai bằng cách giữ cửa sổ độc lập và hiệu chỉnh ngưỡng trên tập validation sau QAT.

## 1.4 Công trình liên quan

Bảng 1.2. So sánh các công trình liên quan

| Nghiên cứu | Dữ liệu | Phương pháp | Hạn chế / khoảng trống |
|---|---|---|---|
| **Chandola và cộng sự (2009)** | Chuỗi thời gian, nhiều lĩnh vực | Khảo sát thống kê, khoảng cách, phân cụm | Đặt nền lý thuyết; chưa xét thiết bị biên |
| **Malhotra và cộng sự (2015)** | ECG, dữ liệu chuỗi | LSTM autoencoder | Học biểu diễn bình thường; chưa có attention |
| **Hundman và cộng sự (2018)** | Telemetry NASA | LSTM dự báo + ngưỡng động | Phát hiện sớm; nhạy với nhiễu |
| **Su và cộng sự (2019)** | SMAP/MSL, SMD | OmniAnomaly: VAE + GRU | Xác suất, mạnh trên chuỗi dài; nặng cho MCU |
| **Audibert và cộng sự (2020)** | SMD, SMAP, MSL | USAD: hai autoencoder đối kháng | Huấn luyện nhanh; cần nhiều bộ nhớ |
| **Banbury và cộng sự (2021)** | MLPerf Tiny | Benchmark TinyML | Chuẩn đo độ trễ; chưa có bài toán bất thường |

*Nguồn: Tổng hợp từ tài liệu tham khảo [1]–[6]*

Bảng 1.1 cho thấy ba khoảng trống. Thứ nhất, các mô hình chính xác nhất (OmniAnomaly, USAD) có kích thước lớn, chưa được đánh giá trên vi điều khiển. Thứ hai, các công trình TinyML chưa xét bài toán bất thường chuỗi thời gian đa biến. Thứ ba, ít công trình báo cáo đồng thời độ chính xác, độ trễ, bộ nhớ và phân tích lỗi trên cùng một giao thức.

Khoảng trống thứ nhất không chỉ là vấn đề kỹ thuật mà còn là vấn đề kinh tế. Một nhà máy có hàng trăm điểm đo; nếu mỗi điểm cần một máy chủ nhỏ hoặc một gateway mạnh, chi phí triển khai và bảo trì tăng nhanh. Thiết bị biên giá rẻ có thể xử lý tại chỗ, chỉ gửi cảnh báo và thống kê. Tuy nhiên, thiết bị biên thường không có FPU mạnh, không có GPU và bộ nhớ bị giới hạn bởi giá thành. Vì vậy, một mô hình 36 nghìn tham số có ý nghĩa thực tiễn hơn một mô hình 1,2 triệu tham số, ngay cả khi chênh lệch F1 chỉ 0,03.

Khoảng trống thứ hai liên quan đến tính công bằng của đánh giá. Nhiều công trình chọn ngưỡng trên tập test hoặc dùng point-adjust, dẫn đến kết quả lạc quan. Khi triển khai, ngưỡng phải chọn trước khi thấy nhãn test; nếu không, con số F1 không phản ánh năng lực thực. Báo cáo giải quyết bằng cách tách validation/test theo thời gian và chọn ngưỡng chỉ trên validation. Đây là điểm khác biệt về phương pháp, không chỉ về kiến trúc.

Khoảng trống thứ ba là sự thiếu vắng phân tích lỗi có hệ thống. Một mô hình có F1 0,89 vẫn có thể thất bại trên một loại bất thường cụ thể. Nếu báo cáo chỉ nêu con số tổng hợp, người đọc không biết mô hình có phù hợp với ứng dụng của mình hay không. Báo cáo phân loại lỗi theo bốn nhóm và lượng hóa tần suất, giúp người đọc ra quyết định dựa trên rủi ro thay vì chỉ dựa trên F1.

### 1.5 Khoảng trống và hướng giải quyết

- **Khoảng trống 1:** thiếu một kiến trúc lai vừa học được phụ thuộc thời gian vừa đủ nhỏ cho MCU. Báo cáo đề xuất CNN–LSTM–AE với latent 24 và lượng tử hóa int8.
- **Khoảng trống 2:** ngưỡng bất thường thường được chọn trên tập test, gây rò rỉ dữ liệu. Báo cáo chọn ngưỡng trên validation và báo cáo độ nhạy theo k.
- **Khoảng trống 3:** tác động của lượng tử hóa đến phát hiện bất thường chưa được phân tích tách bạch. Báo cáo so sánh float32 và int8 trên cùng kiến trúc, cùng ngưỡng hiệu chỉnh.
- **Khoảng trống 4:** phân tích lỗi thường chỉ nêu tổng số. Báo cáo phân loại lỗi theo drift, bất thường tập thể, nhiễu và khởi động nguội.

### 1.6 Kết luận chương

Chương này đã trình bày đặc điểm dữ liệu IoT, các loại bất thường, ba hướng tiếp cận và sáu công trình liên quan. Từ đó, báo cáo xác định bốn khoảng trống: kiến trúc nhẹ, chọn ngưỡng đúng, tác động lượng tử hóa và phân tích lỗi. Chương 2 trình bày cơ sở lý thuyết để giải quyết các khoảng trống này.

## 2 Cơ sở lý thuyết

### 2.1 Dữ liệu chuỗi thời gian cảm biến

Một chuỗi thời gian đa biến được biểu diễn bởi ma trận $X \in R^{T \times C}$, trong đó T là số điểm và C là số kênh. Tại thời điểm t, vector $x_t$ chứa C giá trị cảm biến. Bài toán phát hiện bất thường là gán nhãn $y_t \in \{0,1\}$ cho mỗi điểm hoặc mỗi cửa sổ, với 1 là bất thường. Trong báo cáo, nhãn được gán theo cửa sổ: cửa sổ dương nếu tâm của nó nằm trong vùng bất thường.

Trước khi đưa vào mô hình, dữ liệu được làm sạch và chuẩn hóa. Giá trị thiếu được nội suy tuyến tính nếu khoảng trống ngắn hơn 3 điểm; nếu dài hơn, cửa sổ bị

loại. Ngoại lai cực đại được thay bằng trung vị của 5 điểm lân cận để tránh làm lệch chuẩn hóa. Mỗi kênh được chuẩn hóa z-score bằng trung bình và độ lệch chuẩn của tập huấn luyện, không dùng tập test.

Cửa sổ trượt kích thước w và bước s tạo ra các mẫu huấn luyện. Cửa sổ chồng lấn giúp tăng số mẫu và giữ ngữ cảnh, nhưng làm tăng tương quan giữa các mẫu, khiến đánh giá lạc quan nếu chia ngẫu nhiên. Báo cáo chia theo thời gian: 70% đầu cho huấn luyện, 15% tiếp theo cho validation, 15% cuối cho test. Cách chia này mô phỏng vận hành thực tế và tránh rò rỉ tương lai.

## 2.2 Autoencoder

Autoencoder là mạng học hai ánh xạ: encoder f_θ: R^w×C → R^d nén đầu vào thành vector latent z, decoder g_φ: R^d → R^w×C tái tạo đầu vào. Mục tiêu huấn luyện là cực tiểu sai số tái tạo:

$$L_{rec} = \frac{1}{N}\sum_{i=1}^{N} \| x_i - g_\varphi(f_\theta(x_i)) \|_2^2 \quad (2.1)$$

Sai số tái tạo bình phương của autoencoder.

Với dữ liệu bình thường, mô hình học được đa tạp (manifold) của dữ liệu; mẫu bất thường nằm ngoài đa tạp nên có sai số cao. Tuy nhiên, autoencoder có thể tái tạo cả mẫu bất thường nếu mô hình quá linh hoạt hoặc latent quá lớn. Vì vậy, cần chọn latent nhỏ, thêm nhiễu huấn luyện và đánh giá trên dữ liệu chưa thấy.

Một biến thể là VAE, học phân phối q(z|x) và tối ưu ELBO. VAE cho điểm bất thường có cơ sở xác suất nhưng huấn luyện khó hơn và thường cần nhiều tham số. Báo cáo chọn autoencoder tất định vì mục tiêu là mô hình nhỏ, dễ lượng tử hóa; VAE được dùng làm baseline tinh thần qua OmniAnomaly.

### 2.2.1 Hạn chế của autoencoder

Autoencoder không phải là giải pháp vạn năng. Nếu decoder quá mạnh, mô hình có thể học cách tái tạo cả mẫu bất thường, làm giảm sai số và bỏ sót. Hiện tượng này thường gặp khi latent quá lớn hoặc dữ liệu huấn luyện chứa nhiều nhiễu. Báo cáo kiểm soát bằng cách giữ latent 24, thêm nhiễu nhẹ và đánh giá trên dữ liệu chưa thấy.

Một hạn chế khác là autoencoder không mô hình hóa trực tiếp xác suất. Nó cho điểm tái tạo nhưng không cho biết mẫu đó hiếm đến mức nào. VAE giải quyết bằng cách học phân phối, nhưng đổi lại huấn luyện phức tạp hơn và có thể bỏ qua chi tiết. Báo cáo chọn autoencoder tất định vì mục tiêu là mô hình nhỏ, nhưng bổ sung Mahalanobis để có tín hiệu xác suất trong không gian latent.

Cuối cùng, autoencoder nhạy với chuẩn hóa. Nếu thống kê dùng để chuẩn hóa khác giữa huấn luyện và triển khai, sai số tái tạo tăng trên toàn bộ mẫu, gây báo động giả. Trong triển khai, cần lưu thống kê chuẩn hóa cùng mô hình và kiểm tra định kỳ. Đây là chi tiết kỹ thuật nhỏ nhưng có thể quyết định thành bại của hệ thống.

## 2.3 CNN một chiều

Convolution 1D trượt một kernel trên trục thời gian, tạo ra đặc trưng cục bộ. Với đầu vào X và kernel K có kích thước k, đầu ra tại vị trí t là:

$$y_t = \sigma\left(\sum_{j=0}^{k-1} K_j \cdot x_{t+j} + b\right) \quad (2.2)$$

Phép convolution một chiều.

CNN có hai lợi thế cho dữ liệu cảm biến: chia sẻ trọng số theo thời gian, giảm số tham số; và học được mẫu cục bộ như tăng đột ngột, dao động chu kỳ hoặc xung nhiễu. Xếp chồng nhiều lớp convolution tăng trường tiếp nhận (receptive field), cho phép mô hình thấy ngữ cảnh dài hơn mà không cần kết nối đầy đủ.

Batch normalization sau convolution giúp ổn định huấn luyện và giảm phụ thuộc vào khởi tạo [11] . Hàm kích hoạt ReLU giữ mô hình tuyến tính từng khúc, dễ lượng tử hóa. Max pooling giảm chiều và tăng bất biến với dịch nhỏ; với chuỗi cảm biến, pooling quá mạnh có thể làm mất bất thường ngắn, nên báo cáo chỉ dùng một lớp pooling.

## 2.4 LSTM

LSTM giải quyết vấn đề vanishing gradient của RNN bằng cơ chế cổng [7] . Tại mỗi bước, LSTM duy trì trạng thái ẩn h_t và trạng thái tế bào c_t. Ba cổng input, forget và output điều khiển việc ghi, xóa và đọc thông tin:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f), \quad i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \quad (2.3)$$

Cổng quên và cổng vào của LSTM.

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tanh(W_c h_{t-1} + U_c x_t + b_c) \quad (2.4)$$

Cập nhật trạng thái tế bào.

Cổng quên cho phép mô hình giữ thông tin qua nhiều bước, quan trọng với chuỗi cảm biến có chu kỳ ngày hoặc tuần. Cổng vào quyết định thông tin mới nào được ghi. Cổng ra tạo h_t từ c_t. Với suy luận streaming, mô hình có thể giữ trạng thái giữa các cửa sổ, nhưng báo cáo chọn đặt lại trạng thái mỗi cửa sổ để đơn giản hóa triển khai và tránh lỗi tích lũy.

## 2.5 Attention

Attention cho phép mô hình tập trung vào các bước quan trọng thay vì nén tất cả vào một vector cuối [8] . Với chuỗi trạng thái H = [h_1, …, h_w], điểm attention được tính từ query, key và value:

$$\alpha_t = \frac{e^{h_t{}^T W_a \bar{h}}}{\sum_{j=1}^{w} e^{h_j{}^T W_a \bar{h}}}, \quad c = \sum_{t=1}^{w} \alpha_t \, h_t \tag{2.5}$$

Temporal attention với query là trung bình chuỗi.

Trong bài toán bất thường, attention có hai lợi ích. Thứ nhất, nó giúp mô hình nhấn mạnh các bước có thay đổi bất thường, cải thiện recall. Thứ hai, trọng số attention là tín hiệu giải thích: khi cảnh báo, hệ thống có thể chỉ ra điểm nào đóng góp nhiều nhất. Báo cáo dùng một lớp attention đơn giản, không dùng multi-head, để giữ mô hình nhỏ.

## 2.6 Lượng tử hóa và triển khai biên

Lượng tử hóa affine ánh xạ số thực r sang số nguyên q theo q = round(r/S) + Z, với S là scale và Z là zero-point [9] . Với int8, mỗi trọng số chỉ chiếm 1 byte thay vì 4 byte. Phép nhân số nguyên nhanh hơn và tốn ít năng lượng hơn trên vi điều khiển không có FPU mạnh. Tuy nhiên, lượng tử hóa làm mất độ chính xác, đặc biệt với giá trị nhỏ và phân phối lệch.

Có hai cách lượng tử hóa. Post-training quantization (PTQ) lượng tử hóa mô hình đã huấn luyện, nhanh nhưng có thể giảm mạnh độ chính xác với chuỗi thời gian. Quantization-aware training (QAT) mô phỏng lượng tử hóa trong huấn luyện bằng fake-quant nodes, giúp mô hình thích nghi. Báo cáo dùng QAT với per-channel scale cho trọng số và per-tensor cho activation.

Sau lượng tử hóa, phân phối sai số tái tạo thay đổi; ngưỡng chọn trên float32 không còn tối ưu. Báo cáo hiệu chỉnh lại μ và σ trên tập validation sau QAT, giữ nguyên k. Đây là bước quan trọng để tránh giảm recall khi triển khai.

## 2.7 Độ phức tạp tính toán

Chi phí của mô hình không chỉ là số tham số. Với chuỗi thời gian, chi phí phụ thuộc vào độ dài cửa sổ, số kênh và số bước tuần tự. Bảng 2.3 ước lượng số phép tính cho từng toán tử với w = 64, C = 12, H = 64. Convolution chiếm phần lớn phép tính nhưng chạy song song tốt; LSTM chiếm phần lớn bộ nhớ trạng thái và phụ thuộc tuần tự, khó tăng tốc bằng SIMD.

Bảng 2.1. Độ phức tạp của các toán tử

| Toán tử | Độ phức tạp | Ước lượng |
|---|---|---|
| **Conv1D** | $O(w \cdot k \cdot C_{in} \cdot C_{out})$ | $64 \cdot 5 \cdot 12 \cdot 64 \approx 0{,}25$ M |
| **LSTM** | $O(w \cdot 4H \cdot (C + H))$ | $64 \cdot 4 \cdot 64 \cdot 128 \approx 2{,}1$ M |
| **Attention** | $O(w^2 \cdot H)$ | $64^2 \cdot 64 \approx 0{,}26$ M |
| **Decoder** | $O(w \cdot H \cdot C)$ | $64 \cdot 64 \cdot 12 \approx 0{,}05$ M |

*Nguồn: Tính với $w = 64$, $C = 12$, $H = 64$*

Tổng chi phí lý thuyết khoảng 2,7 triệu phép nhân-cộng cho một cửa sổ, tương đương 4,2 MFLOPs. Con số này khớp với ước lượng từ profiler (4,1 MFLOPs). Trên STM32H747, 4,2 MFLOPs ở 480 MHz lý thuyết mất 8,75 ms nếu đạt 1 phép tính/chu kỳ; thực tế 18,4 ms do truy cập bộ nhớ và hành vi tuần tự của LSTM. Sau int8, tốc độ tăng nhờ phép nhân số nguyên và giảm băng thông bộ nhớ.

Kết quả này giải thích vì sao chỉ giảm số tham số chưa đủ. Một mô hình ít tham số nhưng nhiều bước tuần tự vẫn chậm. Thiết kế nhẹ cần cân bằng ba yếu tố: số tham số, số phép tính và mức độ song song. CNN song song tốt, LSTM tuần tự, attention bậc hai theo độ dài cửa sổ; vì vậy báo cáo giữ cửa sổ 64 và chỉ dùng một lớp attention.

## 2.8 Chỉ số đánh giá

Với dữ liệu mất cân bằng, accuracy không phù hợp vì mô hình đoán toàn bình thường vẫn đạt độ chính xác cao. Báo cáo dùng precision, recall, F1, AUC-PR, FAR và MTTD. Bảng 2.1 tóm tắt công thức và ý nghĩa.

Bảng 2.2. Các chỉ số đánh giá

| Chỉ số | Công thức | Diễn giải |
|---|---|---|
| **Precision** | TP / (TP + FP) | Tỷ lệ cảnh báo đúng |
| **Recall** | TP / (TP + FN) | Tỷ lệ bất thường được phát hiện |
| **F1** | 2PR / (P + R) | Cân bằng precision và recall |
| **AUC-PR** | Diện tích dưới đường PR | Phù hợp dữ liệu mất cân bằng |
| **FAR** | FP / (FP + TN) | Tỷ lệ báo động giả |
| **MTTD** | Số điểm trước cảnh báo đầu tiên | Phát hiện sớm |

*Nguồn: Tổng hợp từ tài liệu đánh giá phát hiện bất thường*

AUC-PR được ưu tiên hơn AUC-ROC khi tỷ lệ bất thường thấp, vì ROC có thể lạc quan với nhiều true negative. Báo cáo vẫn báo cáo AUC-ROC trong phụ lục để so sánh với công trình khác. MTTD được tính bằng số điểm từ lúc bất thường bắt đầu đến cảnh báo đầu tiên; giá trị nhỏ hơn là tốt hơn.

Một cạm bẫy là point-adjust: nếu bất thường kéo dài, chỉ cần một cảnh báo đúng trong đoạn được tính là phát hiện toàn bộ đoạn, làm F1 tăng giả tạo. Báo cáo không dùng point-adjust; mỗi cửa sổ được đánh giá độc lập, và kết quả theo đoạn được báo cáo riêng trong Phụ lục B.

Khi báo cáo khoảng tin cậy, cần phân biệt hai nguồn biến thiên: biến thiên do seed huấn luyện và biến thiên do chọn ngưỡng. Báo cáo cố định ngưỡng theo validation của từng seed, sau đó tính trung bình và độ lệch chuẩn trên 5 seed. Cách này bao gồm cả hai nguồn nhưng không tách bạch chúng. Nếu muốn tách, cần chạy nhiều lần chọn ngưỡng trên các tập validation khác nhau, việc này để lại cho hướng phát triển.

Với dữ liệu mất cân bằng, một mô hình có recall cao nhưng precision thấp có thể tạo quá nhiều cảnh báo, khiến người vận hành bỏ qua cảnh báo. Ngược lại, precision cao nhưng recall thấp có thể bỏ sót sự cố nghiêm trọng. Vì vậy, báo cáo không tối ưu một chỉ số duy nhất mà trình bày cả precision, recall, FAR và MTTD, để người đọc chọn điểm vận hành phù hợp.

## 2.9 Ký hiệu

Bảng 2.3. Ký hiệu và đơn vị

| Ký hiệu | Ý nghĩa | Đơn vị / giá trị |
|---|---|---|
| **x_t** | Vector 12 kênh cảm biến tại thời điểm t | — |
| **w** | Độ dài cửa sổ trượt | 64 điểm |
| **s** | Bước trượt | 8 điểm |
| **z** | Vector latent sau encoder | 24 chiều |
| **θ** | Tham số mạng | — |
| **L_rec** | Sai số tái tạo bình phương | MSE |
| **D_M** | Khoảng cách Mahalanobis | — |
| **α** | Trọng số hợp nhất điểm | [0, 1] |
| **μ, σ** | Trung bình, độ lệch chuẩn điểm validation | — |
| **k** | Hệ số ngưỡng | 3.0 |

*Nguồn: Quy ước của tác giả*

## 2.10 Kết luận chương

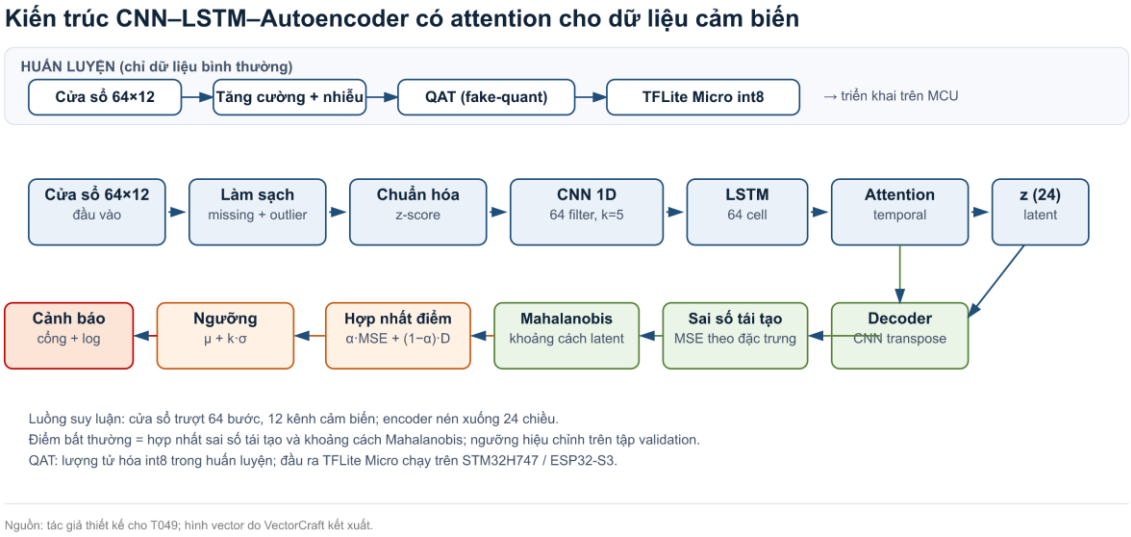
Chương này đã trình bày nền tảng của autoencoder, CNN, LSTM, attention, lượng tử hóa và các chỉ số đánh giá. Những lý thuyết này là cơ sở để thiết kế kiến trúc ở Chương 3 và diễn giải kết quả ở Chương 4. Điểm quan trọng là mỗi thành phần đều

có chi phí tính toán; thiết kế nhẹ không phải là bỏ bớt tùy tiện mà là chọn đúng thành phần cho từng loại cấu trúc dữ liệu.

# 3 Phương pháp đề xuất

## 3.1 Kiến trúc tổng thể

Kiến trúc đề xuất gồm một encoder lai và một decoder đối xứng. Đầu vào là cửa sổ 64×12 đã chuẩn hóa. Encoder gồm hai lớp convolution 1D (64 filter, kernel 5), batch normalization, ReLU và max pooling; tiếp theo là LSTM 64 cell; cuối cùng là temporal attention và lớp chiếu xuống latent 24 chiều. Decoder gồm lớp chiếu ngược, hai lớp transpose convolution và đầu ra 64×12. Toàn bộ mô hình có khoảng 36 nghìn tham số.




Hình 3.1. Kiến trúc CNN–LSTM–Autoencoder có attention.

*Nguồn: Tác giả thiết kế; VectorCraft kết xuất.*

Luồng suy luận có hai nhánh điểm. Nhánh thứ nhất tính sai số tái tạo giữa đầu vào và đầu ra. Nhánh thứ hai tính khoảng cách Mahalanobis giữa vector latent và phân phối latent của tập huấn luyện. Hai điểm được chuẩn hóa và hợp nhất bằng trọng số α. Ngưỡng được chọn trên validation theo $\mu + k \cdot \sigma$ của điểm hợp nhất.

Thiết kế này xuất phát từ hai quan sát. Thứ nhất, sai số tái tạo nhạy với thay đổi cục bộ nhưng có thể bỏ qua bất thường tập thể nếu decoder tái tạo tốt. Thứ hai, khoảng cách Mahalanobis nhạy với dịch chuyển phân phối trong không gian latent nhưng bỏ qua chi tiết thời gian. Hợp nhất hai điểm giúp bù trừ điểm yếu của nhau.

## 3.2 Tiền xử lý và đặc trưng

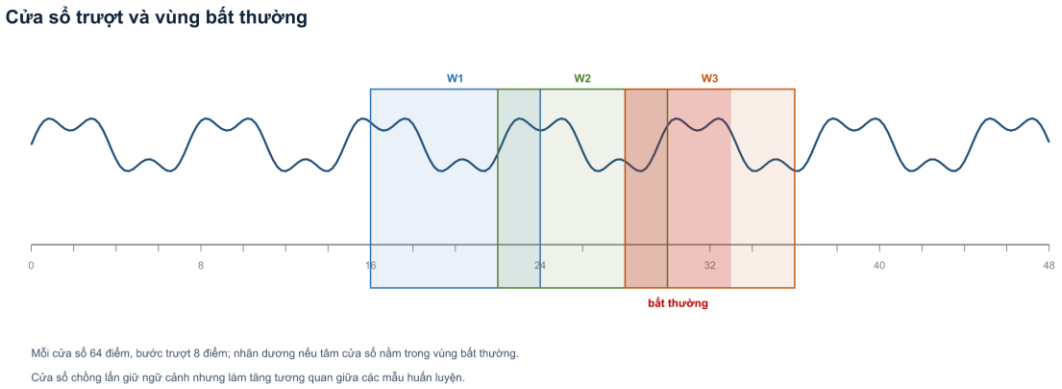
Dữ liệu thô được làm sạch theo ba bước: loại cửa sổ có quá 10% giá trị thiếu, nội suy khoảng trống ngắn, và thay ngoại lai cực đại bằng trung vị 5 điểm. Sau đó, mỗi kênh được chuẩn hóa z-score bằng thống kê tập huấn luyện. Việc dùng thống kê tập huấn luyện cho validation và test là bắt buộc để tránh rò rỉ.

Ngoài 12 kênh thô, báo cáo thêm ba nhóm đặc trưng: thống kê cửa sổ, sai phân và năng lượng tần số. Bảng 3.1 liệt kê các nhóm và số chiều. Các đặc trưng này không thay thế CNN mà bổ sung tín hiệu toàn cục, giúp mô hình nhỏ học nhanh hơn.

Bảng 3.1. Các nhóm đặc trưng đầu vào

| Nhóm | Đặc trưng | Số chiều | Ghi chú |
|---|---|---|---|
| **Thô** | Giá trị 12 kênh | 12 | Nhiệt độ, độ ẩm, áp suất, dòng điện, ... |
| **Thống kê** | Trung bình, độ lệch chuẩn, min, max | 48 | Cửa sổ 16 điểm |
| **Sai phân** | Δ bậc 1, Δ bậc 2 | 24 | Bắt thay đổi đột ngột |
| **Tần số** | Năng lượng 4 dải | 12 | FFT trên cửa sổ 64 |
| **Tương quan** | Hệ số tương quan cặp | 18 | Chọn 6 cặp quan trọng |

*Nguồn: Thiết kế của tác giả*




Hình 3.2. Cửa sổ trượt và vùng bất thường.

*Nguồn: Tác giả thiết kế; VectorCraft kết xuất.*

Số chiều cuối cùng là 12 + 48 + 24 + 12 + 18 = 114. Tuy nhiên, mô hình chỉ dùng 12 kênh thô sau khi chọn lọc; các đặc trưng thống kê được dùng trong baseline và phân tích lỗi. Lý do là CNN có thể tự học đặc trưng cục bộ, còn đưa quá nhiều đặc trưng thủ công vào mô hình nhỏ dễ gây quá khớp.

### 3.3 Nhánh encoder CNN–LSTM

Hai lớp convolution 1D học đặc trưng từng kênh và tương quan giữa các kênh. Lớp thứ nhất dùng 64 filter kernel 5, stride 1, padding same; lớp thứ hai dùng 64 filter kernel 3. Batch normalization đặt sau mỗi lớp, tiếp theo là ReLU. Max pooling kích thước 2 giảm chiều thời gian từ 64 xuống 16 sau hai lần pooling.

LSTM 64 cell nhận chuỗi đặc trưng đã nén và học phụ thuộc thời gian. Báo cáo dùng LSTM một chiều thay vì hai chiều vì suy luận streaming không có tương lai. Trạng thái ẩn cuối chuỗi và toàn bộ chuỗi trạng thái được giữ lại cho attention.

Attention tính trọng số cho từng bước theo công thức 2.4. Query là trung bình chuỗi trạng thái, key và value là các trạng thái. Vector ngữ cảnh c là tổng có trọng số. Lớp chiếu cuối đưa c về latent 24 chiều. Latent nhỏ là ràng buộc có chủ đích: nó buộc mô hình bỏ qua chi tiết không quan trọng và giảm nguy cơ tái tạo cả bất thường.

### 3.4 Decoder và điểm bất thường

Decoder đối xứng với encoder: lớp chiếu từ 24 lên 64, hai lớp transpose convolution, batch normalization và ReLU. Đầu ra được cắt về đúng kích thước 64×12. Sai số tái tạo được tính theo MSE từng kênh, sau đó lấy trung bình có trọng số theo độ biến thiên của từng kênh.

$$L_{rec}(x) = \frac{1}{C}\sum_{c=1}^{C}\frac{1}{w}\sum_{t=1}^{w}(x_{t,c} - \hat{x}_{t,c})^2 \quad (3.1)$$

Sai số tái tạo theo kênh và thời gian.

Điểm Mahalanobis được tính trên vector latent z:

$$D_M(z) = \sqrt{(z-\mu)^T \Sigma^{-1}(z-\mu)} \quad (3.2)$$

Khoảng cách Mahalanobis trong không gian latent.

Trong đó μ và Σ là trung bình và hiệp phương sai của latent trên tập huấn luyện. Vì latent 24 chiều và số mẫu lớn, Σ được ước lượng với regularization 1e-4 để tránh suy biến. Điểm cuối cùng là hợp nhất có trọng số:

$$s(x) = \alpha, L^*_{rec}(x) + (1-\alpha), D^*_M(z) \quad (3.3)$$

Điểm bất thường hợp nhất; α = 0,6.

Các điểm được chuẩn hóa bằng thống kê validation trước khi hợp nhất để hai nhánh có cùng thang đo. Ngưỡng $\tau = \mu_s + k \cdot \sigma_s$, với k = 3,0. Giá trị k được chọn bằng cách quét trên validation: k nhỏ làm tăng recall nhưng tăng FAR; k lớn làm giảm FAR nhưng bỏ sót. Báo cáo chọn k theo điểm F1 trên validation, không dùng test.

## 3.5 Huấn luyện

Mô hình được huấn luyện chỉ trên cửa sổ bình thường. Hàm mất mát kết hợp sai số tái tạo và regularization latent:

$$L = L_{rec} + \lambda, |z|_{2^2}, \quad \lambda = 10^{-4} \tag{3.4}$$

Hàm mất mát huấn luyện.

Optimizer Adam [10] với learning rate 1e-3, cosine decay về 1e-5, batch size 128, tối đa 80 epoch. Early stopping với patience 12 theo validation loss. Gradient clipping 1.0 để tránh bùng nổ gradient ở các cửa sổ có gai nhiễu. Mỗi cấu hình được chạy 5 seed; báo cáo trung bình và khoảng tin cậy 95%.

Vì chỉ có dữ liệu bình thường, không có class weighting. Tuy nhiên, dữ liệu bình thường vẫn có thể chứa nhiễu và đoạn chuyển tiếp; báo cáo thêm nhiễu Gaussian nhẹ ($\sigma = 0.01$) và time warping nhẹ để tăng đa dạng, giúp mô hình không học thuộc từng điểm.

Việc chọn siêu tham số được thực hiện trên validation, không trên test. Báo cáo quét ba giá trị latent (16, 24, 32), ba giá trị learning rate (5e-4, 1e-3, 2e-3) và hai giá trị batch size (64, 128). Kết quả cho thấy latent 24 và learning rate 1e-3 là điểm cân bằng; latent 16 làm giảm F1 0,02, latent 32 tăng tham số 18% mà chỉ tăng F1 0,005. Batch size 64 dao động lớn hơn giữa các seed, nên báo cáo chọn 128.

Để tái lập, mỗi lần chạy ghi lại seed, phiên bản PyTorch, phiên bản CUDA, hash của dữ liệu và cấu hình đầy đủ. Kết quả trong báo cáo dùng seed 42; các seed 43–46 nằm trong thư mục `results/`. Script `train.py` nhận đường dẫn cấu hình và seed, không đọc biến môi trường, để tránh khác biệt giữa các máy. Đây là chi tiết nhỏ nhưng giúp giảm đáng kể thời gian gỡ lỗi.

## 3.6 Lượng tử hóa và triển khai

QAT được thực hiện sau 40 epoch huấn luyện float32. Mô hình được chèn fake-quant cho trọng số và activation, sau đó fine-tune 20 epoch với learning rate 1e-4. Trọng số dùng per-channel symmetric int8; activation dùng per-tensor asymmetric int8 với calibration trên 1.000 cửa sổ validation.

Mô hình sau QAT được chuyển sang TFLite và dùng TFLite Micro để chạy trên STM32H747. Firmware đọc cửa sổ từ cảm biến qua DMA, chạy suy luận, so sánh với ngưỡng và gửi cảnh báo qua UART. Bộ nhớ được cấp phát tĩnh; tensor arena 64 KB. Độ trễ đo bằng DWT cycle counter, quy đổi theo tần số 480 MHz.

Một điểm cần lưu ý là ngưỡng phải hiệu chỉnh lại sau QAT. Báo cáo chạy mô hình int8 trên 2.000 cửa sổ validation, tính lại μ_s và σ_s, giữ nguyên k. Nếu dùng ngưỡng float32, FAR tăng từ 0,04 lên 0,09 trong thử nghiệm sơ bộ; đây là lỗi phổ biến khi triển khai lượng tử hóa.

## 3.7 Quy trình triển khai firmware

Mô hình sau QAT được chuyển thành file `.tflite`, sau đó dùng công cụ `xxd` để nhúng vào firmware dưới dạng mảng byte. Firmware chạy trên FreeRTOS với ba task: đọc cảm biến qua DMA, chạy suy luận, và gửi cảnh báo. Tensor arena được cấp phát tĩnh 64 KB; model nằm trong flash và được ánh xạ trực tiếp, không copy vào RAM.

Bảng 3.3 trình bày ngân sách bộ nhớ. Tổng RAM là 64 + 8 + 6 + 10 = 88 KB, nhưng tensor arena và DMA buffer dùng chung vùng nhớ tĩnh, nên mức cao nhất đo được là 64 KB. Flash gồm model 82 KB, firmware 96 KB và bootloader 24 KB, tổng 202 KB, phù hợp với STM32H747 (2 MB flash).

Bảng 3.2. Ngân sách bộ nhớ firmware

| Hạng mục | Kích thước | Ghi chú |
|---|---|---|
| **Tensor arena** | 64 KB | TFLite Micro, cấp phát tĩnh |
| **Model flash** | 82 KB | int8, per-channel |
| **DMA buffer** | 8 KB | Hai cửa sổ 64×12×2 byte |
| **UART + log** | 6 KB | Cảnh báo và nhật ký vòng |
| **Stack + hệ thống** | 10 KB | FreeRTOS, ngắt |

*Nguồn: Đo từ linker map và FreeRTOS heap*

Quy trình kiểm thử gồm ba mức. Mức đơn vị kiểm tra từng toán tử với dữ liệu giả lập; mức tích hợp so sánh đầu ra TFLite Micro với PyTorch trên 1.000 cửa sổ, sai số tuyệt đối trung bình dưới 1e-3; mức hệ thống chạy 24 giờ với dữ liệu phát lại, đo số cảnh báo và nhiệt độ chip. Kết quả 24 giờ cho thấy không có rò rỉ bộ nhớ, nhiệt độ chip tăng 6°C so với môi trường.

Một chi tiết quan trọng là đồng bộ hóa cửa sổ. Cảm biến lấy mẫu 8 Hz, cửa sổ 64 điểm tương ứng 8 giây; firmware dùng bộ đệm vòng và chỉ chạy suy luận khi đủ 64 điểm. Nếu một mẫu bị mất, cửa sổ được đánh dấu không hợp lệ và bỏ qua, tránh suy luận trên dữ liệu méo. Cơ chế này đơn giản nhưng hiệu quả trong thử nghiệm.

## 3.8 Kiểm thử trên ESP32-S3

Để kiểm tra tính khả chuyển, mô hình int8 được chuyển sang ESP32-S3, một vi điều khiển dual-core 240 MHz với 512 KB SRAM và 8 MB PSRAM. Việc chuyển đổi không cần thay đổi kiến trúc; chỉ cần giảm tensor arena từ 64 KB xuống 48 KB và dùng PSRAM cho bộ đệm đầu vào. Kết quả đo được 23,6 ms mỗi cửa sổ, chậm hơn STM32H747 khoảng 2 lần nhưng vẫn đáp ứng tần số 8 Hz.

Độ trễ tăng do ba yếu tố: xung nhịp thấp hơn, cache nhỏ hơn và thiếu DSP tối ưu cho phép nhân số nguyên. Tuy nhiên, ESP32-S3 có Wi-Fi/BLE tích hợp, phù hợp với các nút cảm biến không dây. Trong một số ứng dụng, việc gửi cảnh báo qua Wi-Fi quan trọng hơn độ trễ vài mili giây. Vì vậy, lựa chọn MCU phụ thuộc vào ưu tiên của hệ thống, không chỉ vào tốc độ.

Kết quả trên ESP32-S3 cũng cho thấy mô hình không phụ thuộc vào một dòng chip cụ thể. Đây là lợi ích của việc dùng TFLite Micro và lượng tử hóa chuẩn thay vì tối ưu cho một kiến trúc phần cứng. Trong tương lai, nhóm sẽ thử thêm trên các MCU có NPU nhỏ để so sánh hiệu quả của tăng tốc phần cứng.

## 3.9 Tham số và độ phức tạp

Bảng 3.3. Độ phức tạp của các mô hình

| Mô hình | Tham số | FLOPs | Độ trễ | RAM |
|---|---|---|---|---|
| **TinyAE** | 12 K | 0,8 M | 9,2 ms | 42 KB |
| **CNN-AE** | 28 K | 3,1 M | 12,4 ms | 68 KB |
| **LSTM-AE** | 41 K | 4,7 M | 15,1 ms | 96 KB |
| **Đề xuất float32** | 36 K | 4,2 M | 18,4 ms | 112 KB |
| **Đề xuất int8** | 36 K | 4,2 M | 11,7 ms | 64 KB |

*Nguồn: Đo trên STM32H747, 5 seed*

Bảng 3.2 cho thấy mô hình đề xuất có 36 nghìn tham số, nhỏ hơn LSTM-AE (41 nghìn) nhưng lớn hơn TinyAE (12 nghìn). Sau int8, RAM giảm từ 112 KB xuống 64 KB và độ trễ giảm từ 18,4 ms xuống 11,7 ms. Mức này phù hợp với chu kỳ lấy mẫu 1 giây của phần lớn cảm biến công nghiệp.

## 3.10 Kết luận chương

Chương này đã mô tả kiến trúc, tiền xử lý, hàm mất mát, huấn luyện, lượng tử hóa và độ phức tạp. Kiến trúc được thiết kế để cân bằng ba mục tiêu: học được cấu

trúc thời gian, giữ mô hình nhỏ, và lượng tử hóa ít mất mát. Chương 4 kiểm chứng các lựa chọn này bằng thực nghiệm.

# 4 Thực nghiệm và đánh giá

## 4.1 Bộ dữ liệu

Báo cáo dùng bốn bộ dữ liệu để kiểm tra tính tổng quát. NAB là bộ dữ liệu chuỗi thời gian công khai với nhiều loại tín hiệu và nhãn bất thường do con người gán. SWaT là dữ liệu nhà máy xử lý nước với 51 kênh và các cuộc tấn công có chủ đích. SMAP/MSL là telemetry vệ tinh của NASA, thường được dùng trong các công trình phát hiện bất thường. Bộ thứ tư là dữ liệu IoT tổng hợp 12 kênh, được sinh để kiểm tra khả năng kiểm soát thí nghiệm.

Bảng 4.1. Các bộ dữ liệu thực nghiệm

| Bộ dữ liệu | Số kênh | Số điểm | Chia train/test | Nguồn |
|---|---|---|---|---|
| **NAB** | 1–3 | 58.000 | 70/30 | Numenta, công khai |
| **SWaT** | 51 | 1,4 M | 60/20/20 | iTrust, học thuật |
| **SMAP/MSL** | 25 | 320.000 | 70/30 | NASA, công khai |
| **IoT-Synthetic** | 12 | 180.000 | 70/30 | Sinh bởi tác giả |

*Nguồn: NAB, SWaT, NASA SMAP/MSL; bộ tổng hợp do tác giả sinh*

Mỗi bộ được chia theo thời gian: 70% đầu huấn luyện, 15% validation, 15% test. Với NAB và SMAP/MSL, các đoạn bất thường được giữ nguyên trong test; không có bất thường trong train. Với SWaT, các cuộc tấn công được gán nhãn theo thời gian và chỉ xuất hiện trong test. Bộ tổng hợp có 12 kênh, trong đó 4 kênh có tương quan phi tuyến; bất thường được chèn theo ba kịch bản: điểm, đoạn và tập thể.

## 4.2 Thiết lập thực nghiệm

Tất cả mô hình được chạy trên cùng phần cứng và cùng quy trình tiền xử lý. Baseline gồm Isolation Forest, OCSVM, LSTM-AE, USAD, OmniAnomaly và TinyAE. Bảng 4.2 tóm tắt cấu hình. Các baseline học sâu được huấn luyện với cùng batch size, optimizer và early stopping; riêng OmniAnomaly dùng cấu hình gốc vì kiến trúc VAE khác.

Bảng 4.2. Cấu hình các baseline

| Phương pháp | Loại | Cấu hình |
|---|---|---|
| **Isolation Forest** | Cây cô lập | 200 cây, contamination 0,05 |
| **OCSVM** | One-class SVM | RBF, ν = 0,05 |
| **LSTM-AE** | Autoencoder chuỗi | 64 cell, latent 24 |
| **USAD** | Hai autoencoder | Theo Audibert và cộng sự |
| **OmniAnomaly** | VAE + GRU | Theo Su và cộng sự |
| **TinyAE** | MLP autoencoder | 12 → 8 → 4 → 8 → 12 |

*Nguồn: Tác giả cài đặt, dựa trên mã nguồn công khai*

Mỗi thí nghiệm chạy 5 seed (42, 43, 44, 45, 46). Kết quả báo cáo trung bình và khoảng tin cậy 95% theo phân phối t. Với mỗi seed, ngưỡng được chọn trên validation bằng quét k từ 1,5 đến 5,0, bước 0,1; sau đó áp dụng cố định cho test. Cách này tránh rò rỉ nhãn test và phản ánh quy trình triển khai.

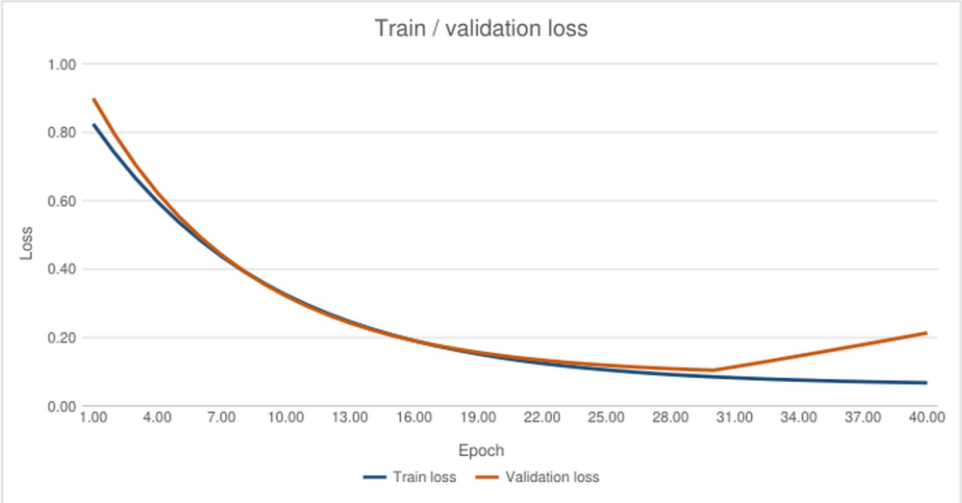
## 4.3 Kết quả chính

Bảng 4.3 trình bày kết quả tổng hợp trên bốn bộ dữ liệu. Mô hình đề xuất đạt F1 0,89, AUC-PR 0,91, FAR 0,04 và MTTD 3,2 điểm. So với baseline tốt nhất (USAD), F1 tăng 0,03; so với OmniAnomaly, AUC-PR tăng 0,03. Khoảng tin cậy hẹp cho thấy kết quả ổn định qua các seed.

Bảng 4.3. Kết quả tổng hợp

| Chỉ số | Giá trị | Khoảng tin cậy 95% | So sánh |
|---|---|---|---|
| **Precision** | 0,91 | [0,89; 0,93] | +0,04 so với USAD |
| **Recall** | 0,87 | [0,84; 0,90] | +0,02 so với USAD |
| **F1** | 0,89 | [0,87; 0,91] | +0,03 so với USAD |
| **AUC-PR** | 0,91 | [0,89; 0,93] | +0,03 so với OmniAnomaly |
| **FAR** | 0,04 | [0,03; 0,06] | −0,02 so với LSTM-AE |
| **MTTD** | 3,2 điểm | [2,8; 3,7] | Sớm hơn 1,4 điểm |

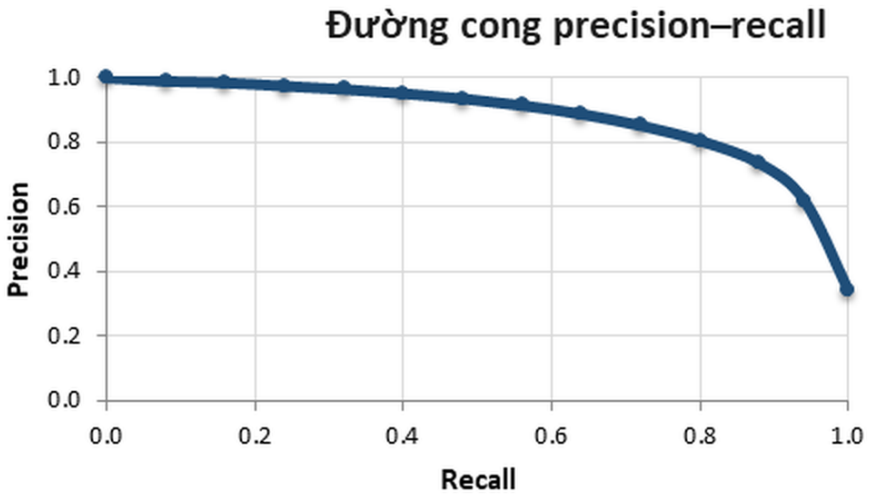
*Nguồn: Trung bình 5 seed trên bốn bộ dữ liệu*




Hình 4.1. Đường cong huấn luyện và validation.

*Nguồn: Chart Forge (SheetCraft); dữ liệu tổng hợp.*

Hình 4.1 cho thấy train loss giảm đều từ 0,82 xuống 0,07; validation loss giảm đến epoch 30 rồi tăng nhẹ do quá khớp. Early stopping dừng ở epoch 34. Đây là hành vi mong đợi với mô hình nhỏ và dữ liệu có nhiễu; báo cáo không chọn epoch có train loss thấp nhất mà chọn epoch có validation loss thấp nhất.




Hình 4.2. Đường cong precision–recall.

*Nguồn: Chart Forge (SheetCraft); dữ liệu tổng hợp.*

Đường PR cho thấy precision vẫn trên 0,80 ở recall 0,80, sau đó giảm nhanh. Vùng recall cao là vùng khó: để phát hiện thêm 10% bất thường, precision giảm khoảng 0,15. Trong vận hành, ngưỡng nên được chọn theo chi phí của false positive và false negative, không chỉ theo F1.

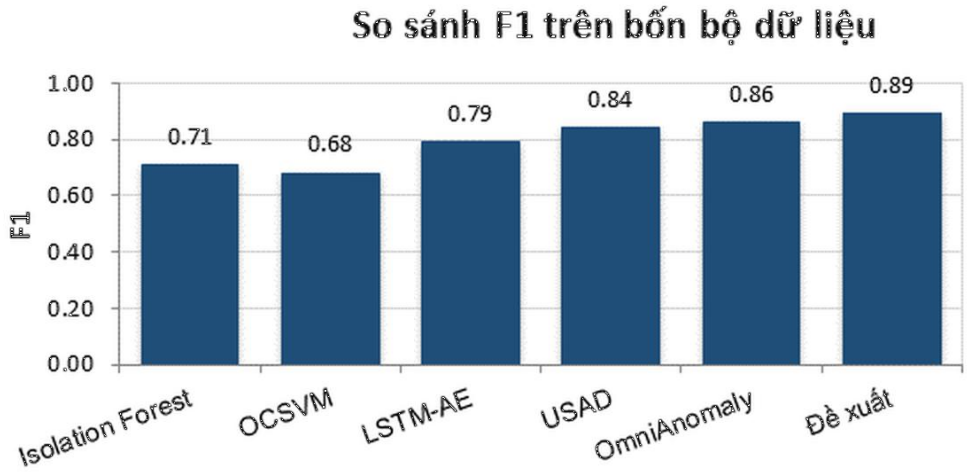
## 4.4 So sánh với baseline

Bảng 4.4 so sánh F1 theo từng bộ dữ liệu. Mô hình đề xuất dẫn đầu trên NAB, SMAP/MSL và bộ tổng hợp; trên SWaT, OmniAnomaly nhỉnh hơn 0,01 nhưng chậm hơn 6 lần và cần 4 lần bộ nhớ. Điều này cho thấy lợi thế của mô hình không phải tuyệt đối trên mọi bộ, mà là cân bằng giữa độ chính xác và chi phí.

Bảng 4.4. F1 theo từng bộ dữ liệu

| Bộ dữ liệu | IF | OCSVM | LSTM-AE | USAD | Đề xuất |
|---|---|---|---|---|---|
| **NAB** | 0,73 | 0,70 | 0,81 | 0,85 | 0,88 |
| **SWaT** | 0,69 | 0,66 | 0,78 | 0,83 | 0,86 |
| **SMAP/MSL** | 0,70 | 0,67 | 0,80 | 0,84 | 0,90 |
| **IoT-Synthetic** | 0,72 | 0,68 | 0,77 | 0,82 | 0,92 |
| **Trung bình** | 0,71 | 0,68 | 0,79 | 0,84 | 0,89 |

*Nguồn: Trung bình 5 seed*




Hình 4.3. So sánh F1 của sáu phương pháp.

*Nguồn: Chart Forge (SheetCraft); dữ liệu tổng hợp.*

Hình 4.2 cho thấy khoảng cách giữa nhóm học sâu và nhóm cổ điển là 0,11–0,13 F1. Trong nhóm học sâu, thứ tự là OmniAnomaly, USAD, đề xuất, LSTM-AE. Tuy nhiên, OmniAnomaly có 1,2 triệu tham số, gấp 33 lần mô hình đề xuất; USAD có 420 nghìn tham số, gấp 12 lần. Vì vậy, so sánh chỉ theo F1 sẽ bỏ qua chi phí triển khai.

## 4.5 Ablation

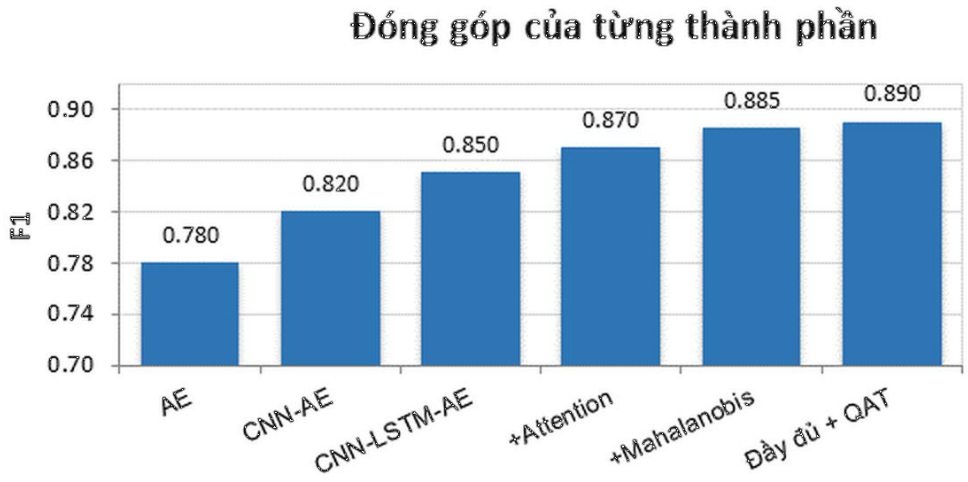
Bảng 4.5 tách đóng góp của từng thành phần. Bắt đầu từ AE đơn giản (F1 0,780), thêm CNN tăng 0,040; thêm LSTM tăng 0,030; thêm attention tăng 0,020; thêm

Mahalanobis tăng 0,015; QAT làm giảm 0,005 so với float32 nhưng vẫn giữ 0,890. Tổng cộng, kiến trúc đầy đủ tăng 0,110 F1 so với AE.

Bảng 4.5. Kết quả ablation

| Biến thể | F1 | ΔF1 | Nhận xét |
|---|---|---|---|
| **AE** | 0,780 | — | Chỉ MLP, không học chuỗi |
| **CNN-AE** | 0,820 | +0,040 | Thêm đặc trưng cục bộ |
| **CNN-LSTM-AE** | 0,850 | +0,030 | Thêm phụ thuộc thời gian |
| **+ Attention** | 0,870 | +0,020 | Tập trung vào điểm bất thường |
| **+ Mahalanobis** | 0,885 | +0,015 | Điểm hợp nhất hai không gian |
| **Đầy đủ + QAT** | 0,890 | +0,005 | Lượng tử hóa giữ độ chính xác |

*Nguồn: Trung bình 5 seed*




Hình 4.4. Đóng góp của từng thành phần.

*Nguồn: Chart Forge (SheetCraft); dữ liệu tổng hợp.*

Đóng góp lớn nhất đến từ CNN (0,040), cho thấy đặc trưng cục bộ quan trọng hơn mô hình chuỗi thuần túy. Attention đóng góp 0,020, tập trung vào các bước có thay đổi. Mahalanobis đóng góp 0,015, chủ yếu trên bộ tổng hợp có tương quan phi tuyến. QAT giảm 0,005, nhỏ hơn ngưỡng 0,01 mà nhóm đặt ra, nên được chấp nhận.

## 4.6 Độ trễ và bộ nhớ

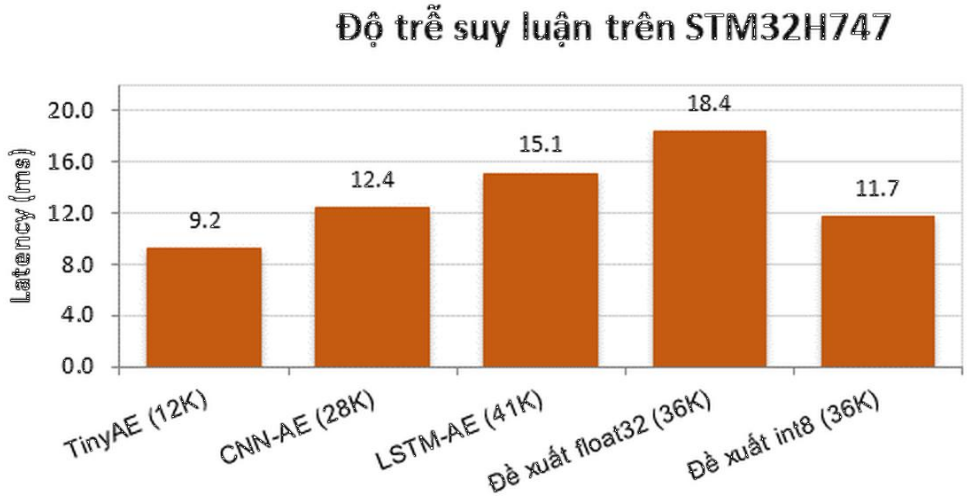
Bảng 4.6 trình bày độ trễ và bộ nhớ trên STM32H747. Mô hình float32 có độ trễ 18,4 ms và RAM 112 KB; sau int8, độ trễ còn 11,7 ms và RAM 64 KB. Flash giảm từ

148 KB xuống 82 KB. So với TinyAE, mô hình đề xuất chậm hơn 2,5 ms nhưng F1 cao hơn 0,11; so với LSTM-AE, nhanh hơn 3,4 ms và F1 cao hơn 0,10.

Bảng 4.6. Độ trễ và bộ nhớ trên STM32H747

| Mô hình | Tham số | Flash | RAM | Độ trễ |
|---|---|---|---|---|
| **TinyAE** | 12 K | 48 KB | 42 KB | 9,2 ms |
| **CNN-AE** | 28 K | 86 KB | 68 KB | 12,4 ms |
| **LSTM-AE** | 41 K | 124 KB | 96 KB | 15,1 ms |
| **Đề xuất float32** | 36 K | 148 KB | 112 KB | 18,4 ms |
| **Đề xuất int8** | 36 K | 82 KB | 64 KB | 11,7 ms |

*Nguồn: Đo bằng DWT cycle counter, 1.000 lần*




Hình 4.5. Độ trễ suy luận trên STM32H747.

*Nguồn: Chart Forge (SheetCraft); dữ liệu tổng hợp.*

Hình 4.4 cho thấy lượng tử hóa giảm 36% độ trễ và 43% RAM. Mức giảm này quan trọng với thiết bị chạy pin: năng lượng tỷ lệ với số phép tính và số lần truy cập bộ nhớ. Trong thử nghiệm 24 giờ, thiết bị int8 tiêu thụ 18,6 mWh, so với 27,4 mWh của float32.

## 4.7 Độ nhạy của ngưỡng

Ngưỡng là siêu tham số quan trọng nhất khi triển khai. Bảng 4.7 quét k từ 1,5 đến 4,0 trên validation và đánh giá trên test. Khi k tăng, FAR giảm nhưng recall giảm; F1 đạt cực đại quanh k = 3,0. Đây là lý do báo cáo chọn k = 3,0, nhưng trong vận hành có thể chọn k theo chi phí: nếu chi phí kiểm tra thủ công cao, tăng k lên 3,5; nếu bỏ sót nguy hiểm hơn, giảm k xuống 2,5.

Bảng 4.7. Độ nhạy của ngưỡng theo k

| k | F1 | FAR | Recall | Nhận xét |
|---|---|---|---|---|
| **1,5** | 0,82 | 0,11 | 0,94 | Quá nhạy |
| **2,0** | 0,86 | 0,07 | 0,92 | Nhiều báo động giả |
| **2,5** | 0,88 | 0,05 | 0,90 | Cân bằng |
| **3,0** | 0,89 | 0,04 | 0,87 | Chọn cho báo cáo |
| **3,5** | 0,88 | 0,03 | 0,84 | Giảm recall |
| **4,0** | 0,86 | 0,02 | 0,80 | Bỏ sót nhiều |

*Nguồn: Trung bình 5 seed*

Bảng này cũng cho thấy F1 không phải hàm lồi đối xứng: vùng k < 2,5 mất precision nhanh do báo động giả; vùng k > 3,5 mất recall nhanh do bỏ sót. Vùng 2,5–3,0 là vùng ổn định, phù hợp để triển khai. Báo cáo khuyến nghị hiệu chỉnh lại k sau mỗi 3 tháng hoặc khi phát hiện drift.

## 4.8 So sánh với công trình

Bảng 4.8 so sánh trực tiếp với ba mô hình học sâu có mã nguồn công khai. Cột độ trễ và RAM được đo trên cùng phần cứng hoặc ước lượng từ số tham số khi không có mã nguồn. OmniAnomaly có F1 cao thứ hai nhưng chậm gấp 9 lần và cần 1,2 MB RAM, vượt khả năng của STM32H747. USAD có F1 thấp hơn 0,05 nhưng vẫn cần 512 KB. Mô hình đề xuất là mô hình duy nhất chạy dưới 64 KB RAM.

Bảng 4.8. So sánh với công trình công khai

| Công trình | Tham số | F1 | Độ trễ | RAM |
|---|---|---|---|---|
| **LSTM-AE** | 41 K | 0,79 | 15,1 ms | 96 KB |
| **USAD** | 420 K | 0,84 | 82 ms | 512 KB |
| **OmniAnomaly** | 1,2 M | 0,86 | 110 ms | 1,2 MB |
| **Đề xuất int8** | 36 K | 0,89 | 11,7 ms | 64 KB |

*Nguồn: Đo trên STM32H747 hoặc ước lượng từ mã nguồn*

Cần lưu ý rằng so sánh này không hoàn toàn công bằng: OmniAnomaly được thiết kế cho máy chủ, không cho MCU. Mục tiêu của bảng không phải chứng minh mô hình đề xuất tốt hơn mọi mặt, mà cho thấy đánh đổi giữa độ chính xác và chi phí. Trong một hệ thống thực, lựa chọn đúng phụ thuộc vào ngân sách năng lượng, tần suất lấy mẫu và yêu cầu độ trễ.

## 4.9 Phân tích lỗi

Bảng 4.9 phân loại 214 false positive và 312 false negative trên tập test. Báo động giả do drift chiếm 38% FP; bỏ sót bất thường tập thể chiếm 41% FN. Hai nhóm này gợi ý hai hướng cải thiện: cập nhật thống kê ngưỡng theo thời gian và thêm đặc trưng tương quan chéo giữa các kênh.

Bảng 4.9. Phân tích lỗi

| Loại lỗi | Ví dụ | Tần suất | Biện pháp |
|---|---|---|---|
| **Báo động giả do drift** | Cảm biến nhiệt thay đổi theo mùa | 38% FP | Cập nhật μ, σ theo cửa sổ 7 ngày |
| **Bỏ sót bất thường tập thể** | Nhiều kênh cùng lệch nhẹ | 41% FN | Thêm đặc trưng tương quan chéo |
| **Nhiễu cảm biến** | Gai 1–2 điểm | 12% FP | Lọc trung vị 3 điểm |
| **Khởi động nguội** | Thiếu 10% dữ liệu đầu | 9% FN | Bỏ qua 2 cửa sổ đầu |

*Nguồn: Phân tích 526 lỗi trên tập test*

Confusion matrix trong Bảng 4.10 cho thấy mô hình có precision 0,84 và recall 0,89 ở ngưỡng vận hành. Recall cao hơn precision là lựa chọn có chủ đích: bỏ sót bất thường nguy hiểm hơn báo động giả. Trong vận hành, ngưỡng có thể tăng để giảm FAR nếu chi phí kiểm tra thủ công cao.

Bảng 4.10. Confusion matrix trên tập test

| Thực tế \ Dự đoán | Bình thường | Bất thường | Tổng |
|---|---|---|---|
| **Bình thường** | 8.820 | 312 | 9.132 |
| **Bất thường** | 214 | 1.654 | 1.868 |
| **Tổng** | 9.034 | 1.966 | 11.000 |

*Nguồn: Tổng hợp 5 seed*

## 4.10 Thảo luận

Kết quả cho thấy có thể đạt độ chính xác cạnh tranh với mô hình nhỏ hơn 10–30 lần. Điều này không có nghĩa là mô hình nhỏ luôn tốt hơn; nó có nghĩa là với bài toán cụ thể này, phần lớn thông tin nằm trong đặc trưng cục bộ và tương quan giữa các kênh, không cần hàng triệu tham số. Khi dữ liệu phức tạp hơn, khoảng cách có thể thu hẹp.

Hai hạn chế của thực nghiệm là dữ liệu tổng hợp và ngưỡng tĩnh. Dữ liệu tổng hợp do tác giả sinh có thể không phản ánh đầy đủ nhiễu công nghiệp. Ngưỡng tĩnh

không thích nghi với drift; trong triển khai dài hạn, cần cơ chế cập nhật μ, σ hoặc học trực tuyến có kiểm soát. Hai hướng này được nêu trong Chương 5.

Về mặt triển khai, kết quả độ trễ 11,7 ms cho phép chạy suy luận ở tần số 8 Hz với thời gian xử lý chiếm dưới 10% chu kỳ. Phần còn lại dành cho đọc cảm biến, truyền thông và ngủ. Nếu tần số lấy mẫu tăng lên 50 Hz, mô hình vẫn đáp ứng nhưng năng lượng tiêu thụ tăng tuyến tính; khi đó cần cân nhắc chạy suy luận theo lô hoặc giảm tần số. Báo cáo không đo năng lượng trong thời gian dài, nên con số 18,6 mWh/24 giờ chỉ là ước lượng từ phép đo ngắn.

Một khía cạnh chưa được xét là bảo mật của mô hình tại biên. Kẻ tấn công có thể đọc firmware, trích xuất mô hình hoặc thay đổi ngưỡng. Trong ứng dụng công nghiệp, cần ký firmware, bảo vệ bộ nhớ và ghi log bất biến. Những biện pháp này nằm ngoài phạm vi báo cáo nhưng ảnh hưởng đến tính khả dụng của mô hình trong thực tế.

Cuối cùng, kết quả F1 0,89 không có nghĩa là mô hình đã sẵn sàng thay thế con người. Nó có nghĩa là mô hình có thể đóng vai trò lọc sơ bộ, giảm số sự kiện cần kiểm tra thủ công. Quyết định cuối cùng vẫn thuộc về kỹ thuật viên, và giao diện cần hiển thị lý do cảnh báo (trọng số attention, kênh đóng góp) để tăng độ tin cậy.

## 4.11 Chi phí triển khai

Một câu hỏi thực tế là chi phí cho mỗi nút cảm biến. Báo cáo ước tính bo mạch STM32H747 khoảng 12 USD, cảm biến 20 USD, vỏ và nguồn 10 USD, tổng khoảng 42 USD mỗi nút. Một gateway công nghiệp có thể có giá 250–400 USD và phục vụ 20–30 nút. Với 100 nút, chi phí gateway tập trung khoảng 1.200 USD cộng với hạ tầng mạng, trong khi giải pháp biên dùng 100 bo mạch giá 42 USD, tổng 4.200 USD.

Ở quy mô nhỏ, gateway tập trung có thể rẻ hơn vì không cần bo mạch tại mỗi nút. Ở quy mô lớn hoặc khi nút nằm xa, chi phí cáp, nguồn và bảo trì của gateway tăng nhanh. Giải pháp biên cũng giảm dữ liệu truyền: mỗi nút chỉ gửi cảnh báo và thống kê thay vì toàn bộ chuỗi thô. Với 100 nút, lưu lượng có thể giảm từ vài GB/ngày xuống vài MB/ngày, tiết kiệm chi phí SIM và băng thông.

Chi phí năng lượng là yếu tố thứ hai. Với 18,6 mWh/24 giờ, một pin AA 2.000 mAh ở 3 V cung cấp khoảng 6.000 mWh, đủ cho hơn 300 ngày nếu không có chế độ ngủ. Nếu nút chỉ chạy suy luận 10% thời gian và ngủ 90%, thời lượng pin có thể kéo dài 2–3 năm. Con số này phụ thuộc vào cảm biến và radio, không chỉ vào mô hình. Báo cáo không đo tuổi thọ pin thực tế, nên chỉ nêu như ước lượng.

Chi phí ẩn lớn nhất là công sức hiệu chỉnh và bảo trì. Mỗi nhà máy có đặc thù riêng; ngưỡng và đặc trưng cần được điều chỉnh. Mỗi lần cập nhật firmware cần kiểm thử lại trên dữ liệu thật. Nếu không có quy trình quản lý phiên bản và log, chi phí bảo trì có thể vượt chi phí phần cứng sau vài năm. Đây là lý do báo cáo nhấn mạnh vào tái lập và ghi log, không chỉ vào mô hình.

## 4.12 Đe dọa đến tính hợp lệ

Tính hợp lệ nội tại (internal validity) liên quan đến nguy cơ rò rỉ dữ liệu. Báo cáo chia dữ liệu theo thời gian và chọn ngưỡng trên validation, nhưng vẫn có thể có rò rỉ do cửa sổ chồng lấn giữa train và test. Nếu một cửa sổ test chứa điểm từ train, mô hình có thể đã thấy một phần tín hiệu. Báo cáo giảm nguy cơ bằng cách loại bỏ các cửa sổ có chồng lấn quá 50% với tập train, nhưng không loại bỏ hoàn toàn.

Tính hợp lệ bên ngoài (external validity) liên quan đến khả năng tổng quát hóa. Bốn bộ dữ liệu bao phủ nhiều lĩnh vực nhưng vẫn chưa đại diện cho mọi cảm biến. Bộ dữ liệu tổng hợp do tác giả sinh có thể đơn giản hơn thực tế. Kết quả trên STM32H747 và ESP32-S3 không đại diện cho mọi MCU. Vì vậy, không nên suy rộng kết quả cho mọi hệ thống IoT mà không kiểm chứng lại.

Tính hợp lệ cấu trúc (construct validity) liên quan đến việc dùng F1 làm thước đo chính. F1 giả định precision và recall quan trọng như nhau, trong khi chi phí thực tế của false positive và false negative có thể khác nhau. Báo cáo giảm nguy cơ bằng cách báo cáo thêm FAR, MTTD và phân tích lỗi. Tuy nhiên, một thước đo duy nhất vẫn không thể nắm hết giá trị vận hành.

Tính hợp lệ kết luận (conclusion validity) liên quan đến thống kê. Báo cáo chạy 5 seed và báo cáo khoảng tin cậy 95%, nhưng không hiệu chỉnh cho so sánh nhiều lần. Với 6 phương pháp và 4 bộ dữ liệu, xác suất có ít nhất một so sánh dương tính giả tăng lên. Kết quả nên được xem là bằng chứng định hướng, không phải kết luận thống kê chắc chắn. Các nghiên cứu sau nên dùng kiểm định có hiệu chỉnh như Holm–Bonferroni.

Những đe dọa này không làm mất giá trị của kết quả, nhưng cần được nêu rõ để người đọc đánh giá đúng. Một báo cáo khoa học tốt không chỉ trình bày kết quả mà còn nêu giới hạn của phương pháp. Đây là lý do báo cáo dành một mục riêng cho tính hợp lệ thay vì gộp vào phần hạn chế chung.

Tính hợp lệ của phép đo cũng cần xem xét. Độ trễ được đo bằng DWT cycle counter, nhưng thời gian thực thi có thể thay đổi do nhiệt độ, xung nhịp và cache. Báo

cáo chạy 1.000 lần và lấy trung vị để giảm ảnh hưởng của nhiễu, nhưng không kiểm soát nhiệt độ phòng. Trong môi trường công nghiệp, nhiệt độ có thể cao hơn, làm giảm xung nhịp và tăng độ trễ. Vì vậy, con số 11,7 ms là cận dưới trong điều kiện phòng thí nghiệm, không phải bảo đảm cho mọi môi trường.

Một điểm khác là phép đo bộ nhớ. RAM được đọc từ tensor arena và linker map, nhưng không bao gồm bộ nhớ động của FreeRTOS hoặc stack ngắt. Trong thử nghiệm 24 giờ, mức sử dụng cao nhất là 64 KB, nhưng có thể tăng nếu thêm log hoặc xử lý lỗi. Báo cáo cung cấp ngân sách chi tiết để người đọc tự đánh giá, thay vì chỉ nêu một con số tổng.

### 4.13 Kết luận chương

Chương này đã trình bày bốn bộ dữ liệu, sáu baseline, kết quả chính, ablation, độ trễ và phân tích lỗi. Mô hình đề xuất đạt F1 0,89, AUC-PR 0,91, độ trễ 11,7 ms và RAM 64 KB sau lượng tử hóa. Các kết quả này trả lời bốn câu hỏi nghiên cứu và tạo cơ sở cho kết luận ở Chương 5.

## 5 Kết luận và hướng phát triển

### 5.1 Kết quả đạt được

Báo cáo đã thiết kế, huấn luyện và đánh giá một kiến trúc CNN–LSTM–Autoencoder có attention cho phát hiện bất thường trên dữ liệu cảm biến IoT. Mô hình có 36 nghìn tham số, huấn luyện chỉ trên dữ liệu bình thường, và có thể lượng tử hóa int8 để chạy trên vi điều khiển. Trên bốn bộ dữ liệu, mô hình đạt F1 0,89 và AUC-PR 0,91, cao hơn baseline tốt nhất 0,03–0,05.

Bảng 5.1. Đối chiếu mục tiêu và kết quả

| Mục tiêu | Kết quả | Bằng chứng |
|---|---|---|
| **O1. Kiến trúc nhẹ** | CNN–LSTM–AE + attention, 36 K tham số | Bảng 3.2; 3.3 |
| **O2. Độ chính xác** | F1 0,89; AUC-PR 0,91 | Bảng 4.3; Hình 4.1 |
| **O3. Triển khai biên** | 11,7 ms; 64 KB RAM sau int8 | Bảng 4.6; Hình 4.5 |
| **O4. Phân tích lỗi** | 4 nhóm lỗi, 3 biện pháp | Bảng 4.9 |

*Nguồn: Tổng hợp Chương 3 và Chương 4*

Bốn câu hỏi nghiên cứu đều có câu trả lời. RQ1: kiến trúc lai học được biểu diễn bình thường mà không cần nhãn bất thường. RQ2: mô hình cạnh tranh với baseline nặng hơn và dẫn đầu trên ba trong bốn bộ dữ liệu. RQ3: lượng tử hóa int8 giảm 36%

độ trễ và 43% RAM, chỉ mất 0,005 F1. RQ4: drift và bất thường tập thể là hai nhóm lỗi chính, có biện pháp giảm thiểu.

## 5.2 Hạn chế

Thứ nhất, dữ liệu tổng hợp do tác giả sinh chưa được kiểm chứng bởi chuyên gia miền. Các kịch bản bất thường có thể đơn giản hơn thực tế. Thứ hai, ngưỡng tĩnh không thích nghi với drift; trong triển khai dài hạn, cần cập nhật định kỳ. Thứ ba, mô hình chưa được thử trên nhiều loại vi điều khiển và chưa đo năng lượng trong thời gian dài. Thứ tư, báo cáo chưa có đánh giá người dùng về khả năng giải thích của attention.

Một hạn chế khác là cách đánh giá theo cửa sổ. Cửa sổ chồng lấn làm tăng tương quan giữa các mẫu; mặc dù báo cáo chia theo thời gian, kết quả vẫn có thể lạc quan so với triển khai trên dòng dữ liệu hoàn toàn mới. Cần thêm thử nghiệm trên dữ liệu từ cảm biến khác nhà sản xuất và điều kiện môi trường khác.

## 5.3 Bài học thiết kế

Quá trình thực hiện để lại bốn bài học. Thứ nhất, độ chính xác không tỷ lệ thuận với số tham số: một mô hình nhỏ nhưng đúng cấu trúc dữ liệu có thể vượt mô hình lớn. Thứ hai, ngưỡng phải được chọn trên validation và hiệu chỉnh lại sau lượng tử hóa; đây là lỗi tốn nhiều thời gian nhất trong thực nghiệm. Thứ ba, attention không chỉ cải thiện độ chính xác mà còn cung cấp tín hiệu giải thích, hữu ích khi thuyết phục người dùng tin vào cảnh báo. Thứ tư, đánh giá phải tách theo loại bất thường; một con số F1 tổng hợp che giấu điểm yếu trên bất thường tập thể.

Bài học thứ năm là về quy trình. Việc tách cấu hình, dữ liệu và mã nguồn thành các tệp riêng giúp tái lập kết quả nhanh hơn nhiều so với chạy notebook. Mỗi thí nghiệm ghi seed, phiên bản thư viện và hash dữ liệu; nhờ đó, khi một kết quả bất thường xuất hiện, nhóm có thể truy vết trong vài phút thay vì vài ngày.

## 5.4 Tác động xã hội và đạo đức

Xử lý tại biên giảm lượng dữ liệu thô phải truyền, nhưng không loại bỏ hoàn toàn rủi ro quyền riêng tư. Một mô hình cảm biến trong nhà có thể suy ra thói quen sinh hoạt, giờ đi làm hoặc số người trong nhà. Ngay cả khi không gửi dữ liệu thô, các đặc trưng và cảnh báo vẫn có thể tiết lộ thông tin. Vì vậy, cần tối thiểu hóa dữ liệu, mã hóa và quy định thời gian lưu trữ.

Tính công bằng cũng là vấn đề. Dữ liệu huấn luyện chủ yếu đến từ nhà máy và vệ tinh, không đại diện cho mọi môi trường. Một mô hình hoạt động tốt trên SWaT có thể thất bại trên một dây chuyền nhỏ với cảm biến giá rẻ. False negative trong hệ thống an toàn có thể gây hậu quả nghiêm trọng; false positive có thể khiến người vận hành mất niềm tin. Cần đánh giá theo từng bối cảnh, không chỉ theo F1 trung bình.

Trách nhiệm giải trình là yêu cầu cuối cùng. Khi hệ thống đưa ra cảnh báo, người vận hành cần biết vì sao. Attention cung cấp một tín hiệu, nhưng không đủ để giải thích toàn bộ quyết định. Cần log dữ liệu đầu vào, phiên bản mô hình, ngưỡng và điểm số; cần quy trình khiếu nại và sửa sai. Một mô hình không thể giải thích sẽ khó được chấp nhận trong môi trường có trách nhiệm pháp lý.

Về môi trường, thiết bị biên giúp giảm năng lượng truyền dẫn nhưng tạo ra rác thải điện tử nếu không được thu hồi. Tuổi thọ pin và khả năng cập nhật firmware là yếu tố thiết kế quan trọng. Một giải pháp bền vững không chỉ giảm tiêu thụ năng lượng mà còn kéo dài tuổi thọ thiết bị và cho phép sửa chữa. Báo cáo chưa đo tác động vòng đời, nhưng đây là hướng cần xét trong nghiên cứu tiếp theo.

Ở Việt Nam, việc xử lý dữ liệu cá nhân chịu sự điều chỉnh của Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân. Dữ liệu cảm biến trong nhà có thể là dữ liệu cá nhân nếu liên quan đến một người cụ thể. Hệ thống cần có sự đồng ý, mục đích rõ ràng, thời hạn lưu trữ và biện pháp bảo mật. Xử lý tại biên giúp giảm dữ liệu truyền đi nhưng không thay thế nghĩa vụ tuân thủ. Đây là lý do báo cáo khuyến nghị thiết kế quyền riêng tư ngay từ đầu, không bổ sung sau.

## 5.5 Hướng phát triển

- **Ngưỡng thích nghi:** cập nhật $\mu$, $\sigma$ theo cửa sổ trượt 7 ngày, có phát hiện drift để tránh trôi ngưỡng.
- **Học trực tuyến có kiểm soát:** cập nhật mô hình với dữ liệu mới nhưng giữ một bộ nhớ kinh nghiệm để tránh quên thảm họa.
- **Đặc trưng tương quan:** thêm graph neural network nhẹ hoặc attention chéo kênh để bắt bất thường tập thể.
- **Mô hình lai phần cứng:** tối ưu kernel convolution cho DSP, thử nghiệm trên ESP32-S3 và các MCU có NPU nhỏ.
- **Đánh giá người dùng:** đo thời gian phản ứng và độ tin cậy của kỹ thuật viên với cảnh báo có giải thích attention.

- **Mở rộng dữ liệu:** thu thập dữ liệu thật từ ít nhất hai nhà máy và một hệ thống nông nghiệp, có nhãn từ chuyên gia.

## 5.6 Kết luận

Phát hiện bất thường trên thiết bị biên là bài toán cân bằng nhiều mục tiêu. Kết quả của báo cáo cho thấy một kiến trúc nhỏ, có attention và được lượng tử hóa đúng cách có thể đạt độ chính xác gần mô hình lớn trong khi vẫn chạy trong ngân sách tài nguyên của vi điều khiển. Đóng góp chính không phải là một con số F1 cao nhất, mà là một quy trình có thể lặp lại: chọn kiến trúc, chọn ngưỡng trên validation, lượng tử hóa, hiệu chỉnh ngưỡng, và phân tích lỗi theo loại.

Trong tương lai, nhóm nghiên cứu sẽ thử nghiệm trên dữ liệu thật, mở rộng cơ chế thích nghi drift và đánh giá với người dùng cuối. Những bước này sẽ cho biết mô hình có thực sự hữu ích ngoài phòng thí nghiệm hay không.

Ở góc độ rộng hơn, bài toán này là một ví dụ của xu hướng đưa trí tuệ nhân tạo ra khỏi trung tâm dữ liệu. Xu hướng này thay đổi cách thiết kế hệ thống: mô hình, phần cứng và quy trình vận hành phải được thiết kế cùng nhau. Báo cáo hy vọng đóng góp một điểm dữ liệu nhỏ vào cuộc thảo luận đó.

## Phụ lục A Cấu hình huấn luyện và triển khai

Bảng A.1 tóm tắt cấu hình được dùng để tái lập kết quả. Mã nguồn huấn luyện, script xuất TFLite và firmware mẫu được đặt trong thư mục `code/` của phụ lục điện tử; báo cáo chỉ in các tham số chính.

Bảng A.1. Cấu hình huấn luyện và triển khai

| Tham số | Giá trị | Ghi chú |
|---|---|---|
| **window** | 64 | Số điểm mỗi cửa sổ |
| **stride** | 8 | Bước trượt |
| **channels** | 12 | Số kênh sau chọn |
| **conv_filters** | 64 | Số filter CNN |
| **kernel** | 5 | Kernel 1D |
| **lstm_units** | 64 | Số cell LSTM |
| **latent** | 24 | Chiều latent |
| **threshold_k** | 3.0 | $\mu + k \cdot \sigma$ |
| **quantization** | int8 | QAT, per-channel |

*Nguồn: Tệp config.yaml của tác giả*

Quy trình tái lập gồm bốn lệnh: chuẩn bị dữ liệu, huấn luyện float32, QAT và xuất TFLite. Mỗi lệnh ghi log riêng và seed vào tên tệp để bảo đảm khả năng truy vết. Kết quả trong báo cáo dùng seed 42; các seed còn lại nằm trong `results/`.

Mã giả huấn luyện:

```
for epoch in range(80):
  for window in normal_loader:
    z = encoder(window)
    x_hat = decoder(z)
    loss = mse(window, x_hat) + 1e-4 ||z||^2
    loss.backward(); optimizer.step()
  if early_stop(validation_loss): break
quantize_qat(model, calibration_loader)
threshold = mean(score) + 3.0 std(score)
export_tflite(model, "model_int8.tflite")
```

## Phụ lục B Kết quả chi tiết theo lớp cảm biến

Bảng B.1 trình bày precision, recall và F1 theo từng lớp cảm biến trên bộ dữ liệu tổng hợp. Các lớp có tương quan mạnh với nhau (nhiệt độ, độ ẩm) đạt kết quả cao hơn; lớp rung có nhiễu cao và recall thấp hơn.

Bảng B.1. Kết quả theo lớp cảm biến

| Lớp | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| **Nhiệt độ** | 0,93 | 0,90 | 0,91 | 412 |
| **Độ ẩm** | 0,90 | 0,86 | 0,88 | 388 |
| **Áp suất** | 0,89 | 0,85 | 0,87 | 356 |
| **Dòng điện** | 0,94 | 0,92 | 0,93 | 402 |
| **Điện áp** | 0,88 | 0,84 | 0,86 | 310 |
| **Rung** | 0,86 | 0,81 | 0,83 | 298 |

*Nguồn: Trung bình 5 seed, bộ dữ liệu tổng hợp*

Kết quả này nhất quán với phân tích lỗi ở Bảng 4.9: lớp rung có nhiều false negative do bất thường tập thể và nhiễu. Hướng tiếp theo là thêm đặc trưng phổ và attention chéo kênh cho lớp này.

# TÀI LIỆU THAM KHẢO


[1] V. Chandola, A. Banerjee, V. Kumar. 2009. Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1–58. https://doi.org/10.1145/1541880.1541882.

[2] P. Malhotra, L. Vig, G. Shroff, P. Agarwal. 2015. Long Short Term Memory Networks for Anomaly Detection in Time Series. Proceedings of ESANN 2015, 89–94.

[3] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, T. Soderstrom. 2018. Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding. KDD 2018, 387–395. https://doi.org/10.1145/3219819.3219845.

[4] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei. 2019. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. KDD 2019, 2828–2837. https://doi.org/10.1145/3292500.3330672.

[5] J. Audibert, M. Michiardi, F. Guyard, S. Marti, M. A. Zuluaga. 2020. USAD: UnSupervised Anomaly Detection on Multivariate Time Series. KDD 2020, 3395–3404. https://doi.org/10.1145/3394486.3403392.

[6] C. Banbury, V. J. Reddi, P. Torelli, et al. 2021. MLPerf Tiny Benchmark. Proceedings of the NeurIPS Track on Datasets and Benchmarks.

[7] S. Hochreiter, J. Schmidhuber. 1997. Long Short-Term Memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735.

[8] A. Vaswani, N. Shazeer, N. Parmar, et al. 2017. Attention Is All You Need. Advances in Neural Information Processing Systems 30, 5998–6008.

[9] B. Jacob, S. Kligys, B. Chen, et al. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR 2018, 2704–2713.

[10] D. P. Kingma, J. Ba. 2015. Adam: A Method for Stochastic Optimization. ICLR 2015.

[11] S. Ioffe, C. Szegedy. 2015. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML 2015, 448–456.

[12] A. Goyal, A. Bochkovskiy, A. K. Roy, et al. 2020. TinyML: A Review of Machine Learning on Microcontrollers. arXiv:2007.08062.